TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO KẾT THÚC HỌC PHẦN

Lập trình nhúng và thời gian thực

Tên đề tài: Hệ thống bãi đỗ xe thông minh sử dụng ESP32-C3 và ESP8266

Danh sách thành viên:

Nguyễn Chí Minh 24020232

Đinh Đức Thịnh 24020313

Nguyễn Việt Tiến 24020322

Giảng viên: Trần Hoàng Việt

Hà Nội – 2026

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn chân thành tới thầy Trần Hoàng Việt, giảng viên học phần Lập trình nhúng và thời gian thực, đã tận tình giảng dạy và định hướng trong suốt quá trình học tập và thực hiện đề tài. Những kiến thức về hệ điều hành thời gian thực, lập trình ngoại vi và giao tiếp không dây từ học phần là nền tảng để nhóm xây dựng hệ thống bãi đỗ xe thông minh này.

Nhóm cũng cảm ơn Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội đã tạo môi trường học tập và điều kiện để nhóm thực hành, cùng các bạn trong lớp đã góp ý trong quá trình thử nghiệm mô hình. Dù đã cố gắng, báo cáo khó tránh khỏi thiếu sót; nhóm rất mong nhận được ý kiến đóng góp của thầy để hoàn thiện hơn.

Hà Nội, tháng 11 năm 2026

Nhóm sinh viên thực hiện

LỜI CAM ĐOAN

Nhóm xin cam đoan báo cáo và hệ thống được trình bày là kết quả làm việc của các thành viên trong nhóm dưới sự hướng dẫn của thầy Trần Hoàng Việt. Mã nguồn, mô hình phần cứng, số liệu đo và kết quả kiểm thử trong báo cáo là trung thực, do nhóm tự thực hiện. Các tài liệu, thư viện và công cụ của bên thứ ba đều được ghi rõ nguồn trong báo cáo và trong phần tài liệu tham khảo. Nhóm xin chịu hoàn toàn trách nhiệm về lời cam đoan này.

Hà Nội, tháng 11 năm 2026

Nhóm sinh viên thực hiện

TÓM TẮT

Tại các bãi đỗ xe quy mô nhỏ như bãi của chung cư mini, văn phòng hay trường học, việc vận hành phần lớn vẫn thủ công: người lái không biết trước còn chỗ trống hay không, phí gửi xe được tính và thu bằng tay, còn các sự cố cháy hoặc ngập nước thường chỉ được phát hiện khi đã lan rộng. Đề tài xây dựng một hệ thống bãi đỗ xe thông minh nhằm tự động hóa ba công việc này: xác định trạng thái từng chỗ đỗ, kiểm soát xe ra vào kèm tính phí và thanh toán không tiền mặt, cảnh báo sự cố và tự mở lối thoát.

Sản phẩm là một mô hình bãi xe 5 chỗ đỗ, 2 cổng có thanh chắn, điều khiển bởi ba vi điều khiển Wi-Fi. Máy chủ ESP32-C3 chạy hệ điều hành thời gian thực FreeRTOS, quản lý phiên gửi xe, điều khiển thanh chắn và còi, hiển thị trên màn hình TFT tại cổng và phục vụ trang web quản lý tại địa chỉ `parking.local`. Hai bo ESP8266 đóng vai trò trạm thu thập: một bo đọc năm cảm biến hồng ngoại ở các ô đỗ cùng cảm biến lửa và mực nước, bo còn lại đọc thẻ RFID ở hai cổng và cảm biến nhiệt độ. Người lái quẹt thẻ để vào; khi ra, màn hình hiển thị mã VietQR với số tiền đúng bằng phí gửi xe, hệ thống đối soát giao dịch qua dịch vụ SePay và chỉ mở thanh chắn khi đã nhận đủ tiền. Cảm biến hồng ngoại đặt dưới mỗi thanh chắn giữ thanh chắn mở khi xe còn nằm trong cổng. Khi phát hiện lửa, hệ thống bật còi và mở cả hai thanh chắn để sơ tán; khi nước ngập vượt ngưỡng, hệ thống ngừng cho xe vào.

Báo cáo trình bày cơ sở lý thuyết, thiết kế phần cứng kèm thuyết minh mạch, kiến trúc phần mềm, hiện thực theo từng use case và kết quả kiểm thử. Bộ kiểm thử tự động gồm 439 phép kiểm tra chạy trên máy tính, cùng các kịch bản thử trên mô hình thật, đều cho kết quả đạt.

Từ khóa: bãi đỗ xe thông minh, ESP32-C3, ESP8266, FreeRTOS, RFID, VietQR, hệ thống nhúng thời gian thực.

MỤC LỤC

DANH SÁCH THUẬT NGỮ VIẾT TẮT **8**

DANH SÁCH BẢNG BIỂU **9**

DANH SÁCH HÌNH VẼ **11**

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI **12**

1.1 Đặt vấn đề 12

1.2 Tổng quan về hệ thống 12

1.3 Mục tiêu và phạm vi 13

1.4 Khảo sát các giải pháp hiện có 14

1.5 Cấu trúc báo cáo 15

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG **17**

2.1 Hệ thống nhúng và hệ điều hành thời gian thực 17

2.2 Vi điều khiển 18

2.3 Cảm biến và cơ cấu chấp hành 19

2.4 Giao tiếp ngoại vi 20

2.5 Giao thức mạng và thanh toán 20

2.6 Công cụ phát triển 21

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **23**

3.1 Yêu cầu chức năng 23

3.2 Yêu cầu phi chức năng 25

3.3 Kiến trúc tổng thể 26

3.4 Thiết kế phần cứng 28

3.4.1 Bố trí thiết bị trên mô hình 28

3.4.2 Sơ đồ chân 29
3.4.3 Thuyết minh mạch điện 30
3.5 Kiến trúc phần mềm máy chủ 37
3.6 Thiết kế giao thức truyền thông 39

CHƯƠNG 4. HIỆN THỰC HỆ THỐNG THEO USE CASE 41

4.1 UC01. Xem chỗ trống tại cổng 41
4.2 UC02. Gửi xe bằng thẻ RFID 42
4.2.1 Đọc thẻ trên hai đầu đọc dùng chung bus 44
4.2.2 Bảo đảm không mất lượt quẹt thẻ 45
4.2.3 Điều khiển thanh chắn 45
4.2.4 Từ chối lượt quẹt không hợp lệ 47
4.3 UC03. Thanh toán phí và lấy xe 47
4.3.1 Sinh mã VietQR trên vi điều khiển 49
4.3.2 Đối soát giao dịch qua SePay 50
4.4 UC04. Xác nhận thanh toán tại quầy 51
4.5 UC05. Điều khiển thanh chắn từ xa 52
4.6 UC06. Báo cháy và mở lối thoát 53
4.7 UC07. Báo ngập và ngừng nhận xe 54
4.8 UC08. Xem tình trạng bãi trên trang web 55
4.9 UC09. Tra cứu nhật ký và doanh thu 57
4.10 UC10. Hiệu chỉnh danh sách xe trong bãi 59

CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ 60

5.1 Phương pháp kiểm thử 60
5.2 Kết quả kiểm thử tự động 60
5.3 Kết quả kiểm thử trên mô hình 61
5.4 Đánh giá hiệu năng và tài nguyên 63
5.5 Hạn chế còn tồn tại 64

KẾT LUẬN **65**

TÀI LIỆU THAM KHẢO **66**

PHỤ LỤC A. ĐÓNG GÓP CỦA THÀNH VIÊN VÀ PHÂN CHIA ĐIỂM **69**

A.1 Phân công công việc 69

A.2 Tỉ lệ đóng góp và phân chia điểm 70

DANH SÁCH THUẬT NGỮ VIẾT TẮT

Viết tắt	Ý nghĩa
ACK	Acknowledgement, bản tin xác nhận đã nhận
ADC	Analog-to-Digital Converter, bộ chuyển đổi tương tự sang số
API	Application Programming Interface, giao diện lập trình ứng dụng
DMA	Direct Memory Access, truy cập bộ nhớ trực tiếp
GPIO	General Purpose Input/Output, chân vào ra đa dụng
HTTP(S)	HyperText Transfer Protocol (qua TLS)
IR	Infrared, hồng ngoại
JSON	JavaScript Object Notation
LEDC	LED Control, khối phát PWM phần cứng của ESP32
mDNS	Multicast DNS, phân giải tên miền `.local` trong mạng cục bộ
NVS	Non-Volatile Storage, vùng lưu trữ khóa và giá trị trên flash
PWM	Pulse Width Modulation, điều chế độ rộng xung
RFID	Radio Frequency Identification, nhận dạng bằng sóng vô tuyến
RTOS	Real-Time Operating System, hệ điều hành thời gian thực
SPI	Serial Peripheral Interface, giao tiếp ngoại vi nối tiếp
TFT	Thin-Film Transistor, công nghệ màn hình tinh thể lỏng
UC	Use case, ca sử dụng
UDP	User Datagram Protocol
UID	Unique Identifier, mã định danh của thẻ RFID
VietQR	Chuẩn mã QR chuyển khoản liên ngân hàng của NAPAS

DANH SÁCH BẢNG BIỂU

1.1 So sánh đề tài với các nhóm giải pháp đã khảo sát 15

2.1 So sánh các phương án vi điều khiển 18
2.2 Cảm biến, cơ cấu chấp hành và thông số chính 19
2.3 Các giao tiếp ngoại vi được sử dụng 20
2.4 Thành phần phần mềm và vai trò 21

3.1 Tổng hợp các use case 25
3.2 Yêu cầu phi chức năng và giải pháp đáp ứng 26
3.3 Sơ đồ chân của ba bo mạch 30
3.4 Chức năng của các linh kiện nguồn và linh kiện thụ động 37
3.5 Các tác vụ chạy trên máy chủ 38
3.6 API HTTP của máy chủ 40

4.1 Đặc tả UC01. Xem chỗ trống tại cổng 41
4.2 Đặc tả UC02. Gửi xe bằng thẻ RFID 43
4.3 Đặc tả UC03. Thanh toán phí và lấy xe 48
4.4 Cấu trúc chuỗi VietQR do máy chủ sinh 50
4.5 Đặc tả UC04. Xác nhận thanh toán tại quầy 51
4.6 Đặc tả UC05. Điều khiển thanh chắn từ xa 52
4.7 Đặc tả UC06. Báo cháy và mở lối thoát 53
4.8 Đặc tả UC07. Báo ngập và ngừng nhận xe 54
4.9 Đặc tả UC08. Xem tình trạng bãi trên trang web 55
4.10 Đặc tả UC09. Tra cứu nhật ký và doanh thu 57
4.11 Đặc tả UC10. Hiệu chỉnh danh sách xe trong bãi 59

5.1 Kết quả kiểm thử tự động 61

5.2 Kịch bản kiểm thử trên mô hình và kết quả quan sát 62
5.3 Độ trễ của các luồng chính 63
5.4 Hạn chế, nguyên nhân và hướng khắc phục 64

A.1 Phân công công việc 69
A.2 Tỉ lệ đóng góp và tỉ lệ điểm 70

DANH SÁCH HÌNH VẼ

1.1 Mô hình bãi đỗ xe thông minh nhìn từ trên xuống 13

3.1 Sơ đồ use case tổng quát 24
3.2 Kiến trúc triển khai và luồng dữ liệu 27
3.3 Bố trí phần cứng trên mô hình 28
3.4 Đường cấp nguồn của máy chủ và của mỗi client 31
3.5 Sơ đồ nguyên lý máy chủ: ESP32-C3, màn hình ST7789, 2 servo, còi và 2 cảm biến IR ở cổng 32
3.6 Sơ đồ nguyên lý Sensor Client: ESP8266, 5 cảm biến IR ô đỗ, cảm biến lửa và mực nước 33
3.7 Sơ đồ nguyên lý RFID Client: ESP8266, 2 đầu đọc RC522 dùng chung SPI và DHT11 34
3.8 Kiến trúc phân lớp của firmware máy chủ 37
3.9 Tác vụ, tài nguyên dùng chung và cơ chế đồng bộ 39

4.1 Lọc nhiễu tín hiệu cảm biến ô đỗ 42
4.2 Biểu đồ tuần tự gửi xe, xử lý mất gói và lấy xe 44
4.3 Dạng xung điều khiển servo SG90 45
4.4 Máy trạng thái của một thanh chắn 46
4.5 Máy trạng thái của một thẻ RFID 47
4.6 Biểu đồ tuần tự thanh toán VietQR ở cổng ra 49
4.7 Biểu đồ tuần tự cảnh báo cháy 54
4.8 Trang web quản lý: sơ đồ bãi và điều khiển thanh chắn 56
4.9 Trang web quản lý: cảm biến, doanh thu và nhật ký vào ra 58

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Đặt vấn đề

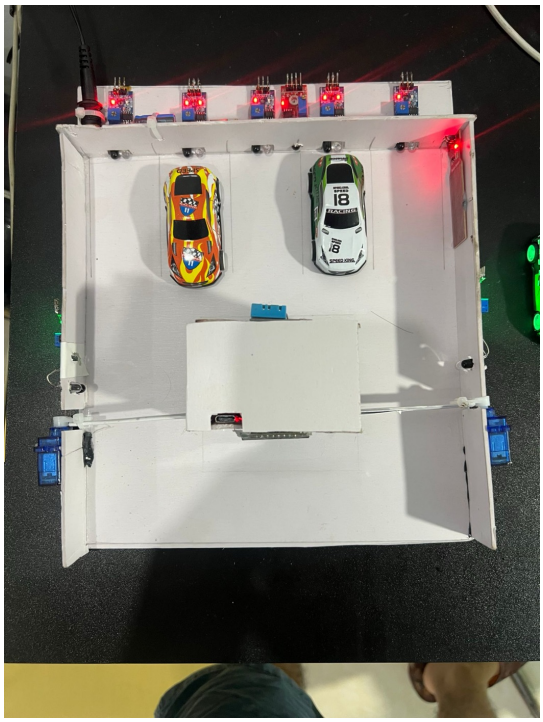
Tại các đô thị lớn, số lượng ô tô tăng nhanh hơn nhiều so với diện tích bãi đỗ. Người lái thường phải đi vòng để tìm chỗ trống, gây ùn tắc cục bộ và lãng phí nhiên liệu. Ở các bãi quy mô nhỏ như bãi của chung cư mini, văn phòng hay trường học, việc vận hành phần lớn vẫn thủ công: vé giấy, ghi giờ bằng tay, tính tiền theo ước lượng của người trông xe và hầu như không có cơ chế cảnh báo khi xảy ra cháy hoặc ngập nước, hai rủi ro thường gặp ở tầng hầm.

Từ thực trạng trên, đề tài đặt ra bài toán: với linh kiện phổ thông giá rẻ, xây dựng một hệ thống tự động hóa trọn vẹn quy trình của một bãi xe nhỏ, từ lúc xe vào cổng, đỗ vào ô, cho tới lúc thanh toán và rời bãi, đồng thời bảo đảm an toàn cho bãi trong suốt thời gian đó. Bài toán này cũng là một ứng dụng điển hình của môn học, đòi hỏi lập trình đa tác vụ trên hệ điều hành thời gian thực, điều khiển ngoại vi bằng phần cứng chuyên dụng và giao tiếp không dây giữa nhiều vi điều khiển.

1.2. Tổng quan về hệ thống

Hệ thống được hiện thực trên một mô hình bãi xe gồm 5 ô đỗ và hai cổng vào, ra có thanh chắn. Hình 1.1 là ảnh chụp mô hình hoàn thiện nhìn từ trên xuống. Dãy ô đỗ nằm dọc vách sau, mỗi ô có một cảm biến hồng ngoại xác định ô đó có xe hay không. Hộp điều khiển ở giữa bãi chứa vi điều khiển trung tâm, màn hình hiển thị quay ra phía người lái và hai đầu đọc thẻ RFID gắn ở hai bên hông, mỗi đầu đọc phục vụ một làn. Hai động cơ servo ở hai góc trước nâng hạ thanh chắn của cổng vào (bên trái) và cổng ra (bên phải).

Từ góc nhìn của người sử dụng, hệ thống vận hành như sau. Khi tới cổng vào, người lái xem số chỗ trống trên màn hình và quẹt thẻ RFID; thanh chắn tự nâng lên, xe đi qua và thanh chắn chỉ hạ xuống khi cảm biến ở cổng xác nhận xe đã ra khỏi vùng thanh chắn. Khi xe đỗ vào một ô, màn hình và trang web quản lý cập nhật trạng thái ô đó trong khoảng một phần mười giây. Khi ra, người lái quẹt cùng thẻ ở cổng ra; màn hình hiển thị phí gửi xe và một mã



Hình 1.1: Mô hình bãi đỗ xe thông minh nhìn từ trên xuống

VietQR mà mọi ứng dụng ngân hàng đều quét được. Ngay khi giao dịch chuyển khoản được xác nhận, thanh chắn cổng ra mở và màn hình hiển thị hóa đơn. Người quản lý theo dõi toàn bộ bãi, nhật ký xe vào ra và doanh thu trên trang web từ điện thoại mà không cần cài ứng dụng. Nếu cảm biến phát hiện lửa, còi báo động kêu và cả hai thanh chắn mở để xe thoát ra; nếu mực nước vượt ngưỡng, hệ thống ngừng nhận xe mới.

1.3. Mục tiêu và phạm vi

Mục tiêu của đề tài được xác định thành các yêu cầu có thể kiểm chứng trên mô hình. Các mục tiêu cụ thể gồm:

- xác định trạng thái từng ô đỗ theo thời gian thực, với độ trễ hiển thị tại bãi dưới 0,2 s và trên trang web khoảng 1 s;
- kiểm soát xe ra vào bằng thẻ RFID, từ chối các lần quẹt sai cổng hoặc khi bãi đầy, và không hạ thanh chắn khi xe còn nằm trong cổng;

– tính phí tự động theo giờ và thu phí bằng chuyển khoản VietQR với số tiền cố định, tự xác nhận giao dịch trước khi mở cổng ra;

– phát hiện cháy và ngập nước, cảnh báo tại chỗ và trên trang web, mở lối thoát khi có cháy;

– cho phép người vận hành quản lý bãi và điều khiển thanh chắn từ xa qua trình duyệt, có xác thực bằng mã PIN cho các thao tác điều khiển.

Phạm vi của đề tài giới hạn ở quy mô mô hình phòng thí nghiệm với xe mô hình và thẻ MIFARE. Các chức năng nhận dạng biển số, thanh toán qua cổng thẻ quốc tế và lưu trữ dữ liệu trên đám mây không thuộc phạm vi đề tài và được nêu trong phần hướng phát triển.

1.4. Khảo sát các giải pháp hiện có

Bãi đỗ xe thông minh không phải là ý tưởng mới. Theo Knowledge Sourcing Intelligence, thị trường toàn cầu đạt khoảng 8,8 tỷ USD năm 2025 và được dự báo đạt 20,4 tỷ USD năm 2030, trong đó khu vực châu Á – Thái Bình Dương tăng trưởng nhanh nhất [16]; Meticulous Research dự báo quy mô 22,9 tỷ USD vào năm 2032 [17]. Các sản phẩm hiện có có thể chia thành ba nhóm theo chức năng chính.

Nhóm thứ nhất là phần mềm quản lý bãi giữ xe như eParking, HTG Parking hay Megaparking. Các phần mềm này ghi nhận lượt xe vào ra, quản lý vé và doanh thu, một số sản phẩm có nhận dạng biển số hoặc dùng thẻ RFID [22], nhưng không biết xe đang đỗ ở vị trí nào. Nhóm thứ hai là hệ thống kiểm soát ra vào trọn gói của các nhà tích hợp như KPS System hay Hikvision, gồm thanh chắn, camera nhận dạng biển số, đầu đọc RFID tầm xa và phần mềm quản lý có kết nối với hệ thống tòa nhà [18], [19]. Nhóm thứ ba là hệ thống phát hiện xe theo từng chỗ đỗ, tiêu biểu là cảm biến không dây Nedap SENSIT và hệ thống dẫn hướng của SKIDATA [20], [21], vốn trả lời câu hỏi chỗ nào còn trống nhưng cần một nền tảng khác để thu phí.

Ở quy mô đô thị, Hà Nội thí điểm ứng dụng iParking từ năm 2017 nhưng dừng chức năng thanh toán tại 12 đơn vị từ tháng 9/2020, một phần do nhân viên nhập sai số xe khiến doanh thu không khớp với thực tế [23], [24]. Trong khi đó, thu phí không dừng đang được mở rộng: từ tháng 5/2024 đã áp dụng tại 5 sân bay lớn [25], và Kế hoạch 198/KH-UBND

yêu cầu từ ngày 1/7/2026, toàn bộ bãi đỗ ô tô tại Hà Nội thu giá dịch vụ theo hình thức điện tử không dừng [26].

Bảng 1.1: So sánh đề tài với các nhóm giải pháp đã khảo sát

Tiêu chí	Kiểm soát ra vào (KPS, Hikvision)	Cảm biến từng chỗ (Nedap, SKIDATA)	Đề tài
Nhận dạng xe	Biển số, RFID tầm xa, QR	Không	RFID tầm gần
Trạng thái từng chỗ	Không có trong tài liệu	Có, là chức năng chính	Có, 5 cảm biến hồng ngoại
Tính phí, thanh toán	Có, nhiều quy tắc giá	Cần hệ thống khác	Tự tính phí, VietQR tự đối soát
Cảnh báo cháy, ngập gắn với thanh chắn	Không có trong tài liệu	Không có trong tài liệu	Có, cháy thì mở cả hai cổng
Hạ tầng	Máy chủ và phần mềm riêng	Nền tảng quản lý riêng	Chạy trên một vi điều khiển
Mức hoàn thiện	Vận hành thực tế nhiều năm	Triển khai quy mô đô thị	Mô hình 5 chỗ, 2 cổng

Bảng 1.1 đối chiếu đề tài với hai nhóm giải pháp có chức năng gần nhất. Phân tích cho thấy ở quy mô lớn, các giải pháp thương mại đã rất hoàn thiện về nhận dạng xe, tốc độ qua cổng và khả năng tích hợp; đề tài không nhằm cạnh tranh ở quy mô đó. Khoảng trống nằm ở các bãi nhỏ: phát hiện chỗ trống, kiểm soát ra vào, thanh toán và cảnh báo an toàn thường thuộc những sản phẩm riêng biệt, phải qua nhà tích hợp và không có giá niêm yết [27]. Bài học từ iParking cũng cho thấy việc phụ thuộc vào người nhập liệu là nguồn sai lệch doanh thu. Đề tài hướng tới khoảng trống này với ba đặc điểm: gộp đủ các chức năng trong một hệ thống chi phí thấp, tính phí và đối soát giao dịch hoàn toàn tự động, và gắn cảnh báo an toàn trực tiếp với cơ cấu thanh chắn. Đổi lại, hệ thống chưa có nhận dạng biển số và đầu đọc tầm xa, nên xe vẫn phải dừng để quẹt thẻ.

1.5. Cấu trúc báo cáo

Ngoài phần mở đầu, báo cáo gồm năm chương và phần kết luận. Chương 2 trình bày cơ sở lý thuyết về hệ thống nhúng thời gian thực, các linh kiện và giao thức được sử dụng.

Chương 3 phân tích yêu cầu dưới dạng use case, thiết kế phần cứng kèm thuyết minh mạch và kiến trúc phần mềm. Chương 4 trình bày quá trình hiện thực theo thứ tự các use case. Chương 5 trình bày kết quả kiểm thử và đánh giá. Phần kết luận tổng kết kết quả và nêu hướng phát triển; phụ lục ghi nhận đóng góp của từng thành viên.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày các khái niệm nền tảng của hệ thống nhúng thời gian thực, đặc điểm của các vi điều khiển, cảm biến và cơ cấu chấp hành được lựa chọn, các giao tiếp ngoại vi, giao thức mạng và công cụ phát triển. Với mỗi thành phần, báo cáo phân tích lý do lựa chọn dựa trên yêu cầu của bài toán.

2.1. Hệ thống nhúng và hệ điều hành thời gian thực

Hệ thống nhúng là hệ thống máy tính chuyên dụng được gắn vào một thiết bị lớn hơn để thực hiện một nhóm chức năng cố định, chạy liên tục với tài nguyên hạn chế và giao tiếp trực tiếp với thế giới thực qua cảm biến và cơ cấu chấp hành. Thành phần trung tâm là vi điều khiển, tích hợp CPU, bộ nhớ và các khối ngoại vi như GPIO, bộ định thời, PWM, SPI, ADC và Wi-Fi trên cùng một chip. Một hệ thống nhúng được gọi là thời gian thực khi tính đúng đắn phụ thuộc cả vào thời điểm có kết quả. Trong đề tài, thanh chắn phải đóng đúng thời hạn, cảnh báo cháy phải mở thanh chắn ngay và màn hình phải phản ánh chỗ trống gần như tức thời. Đây là các ràng buộc thời gian mềm: trễ một chút không gây hỏng hóc nhưng làm giảm chất lượng dịch vụ.

FreeRTOS là nhân hệ điều hành thời gian thực mã nguồn mở được Espressif tích hợp sẵn trong ESP-IDF. Mỗi công việc được tổ chức thành một tác vụ có ngăn xếp và mức ưu tiên riêng; bộ lập lịch ưu tiên có chiếm quyền luôn chạy tác vụ sẵn sàng có ưu tiên cao nhất. Tác vụ chờ sự kiện như gói tin đến hay thông báo từ tác vụ khác không tiêu tốn thời gian CPU. Để chia sẻ dữ liệu an toàn, FreeRTOS cung cấp vùng găng (critical section) cho các đoạn mã rất ngắn, mutex cho tài nguyên bị chiếm giữ lâu như bus SPI, và thông báo tác vụ (task notification) như cơ chế đánh thức nhẹ nhất giữa hai tác vụ. Ngoài ra, bộ hẹn giờ `esp_timer` cho phép gọi một hàm sau một khoảng thời gian với độ phân giải micro giây.

Máy chủ của hệ thống phải đồng thời nhận gói tin, phục vụ trang web, vẽ màn hình và điều khiển thanh chắn; nếu viết trong một vòng lặp duy nhất, một thao tác chậm như vẽ toàn màn hình (khoảng 23 ms) sẽ làm trễ mọi việc khác. FreeRTOS cho phép việc quan trọng như

điều khiển cổng luôn được thực thi trước việc trang trí như ảnh động. Ngược lại, hai bo thu thập chỉ thực hiện tuần tự vài thao tác ngắn, nên một vòng lặp không chặn trên nền Arduino là đủ và đơn giản hơn.

2.2. Vi điều khiển

Hệ thống dùng ba bo mạch với hai loại vi điều khiển. Bảng 2.1 so sánh ESP32-C3, được chọn cho máy chủ, với các phương án thường gặp trong đồ án sinh viên.

Bảng 2.1: So sánh các phương án vi điều khiển

Tiêu chí	ESP32-C3	ESP32	ESP8266	Arduino Uno
CPU	RISC-V 160 MHz	2 nhân 240 MHz	Xtensa 80 MHz	AVR 16 MHz
SRAM	400 KB	520 KB	khoảng 80 KB	2 KB
Wi-Fi	Có	Có	Có	Không
PWM phần cứng	LEDC 14 bit	LEDC	PWM mềm	8 bit
SPI có DMA	Có	Có	Hạn chế	Không
RTOS chính thức	FreeRTOS	FreeRTOS	Hạn chế	Không

Phân tích trong Bảng 2.1 cho thấy yếu tố quyết định là bộ nhớ: riêng ảnh màn hình 240×240 điểm, mỗi điểm 16 bit, đã cần $115\,200$ byte RAM, lớn hơn toàn bộ RAM của ESP8266 hay Arduino Uno. ESP32-C3 SuperMini có đủ RAM, có DMA cho SPI để CPU không phải chờ khi truyền ảnh, có khối PWM phần cứng giữ xung servo ổn định khi CPU bận xử lý mạng, và được hỗ trợ chính thức bởi ESP-IDF cùng FreeRTOS. So với ESP32 bản gốc, ESP32-C3 rẻ và nhỏ hơn (22×18 mm) trong khi một nhân 160 MHz vẫn dư năng lực cho bài toán.

Hai bo thu thập dùng ESP8266 vì nhu cầu tính toán rất thấp: đọc vài chân số, một kênh ADC, giao tiếp SPI với đầu đọc thẻ và gửi gói tin nhỏ. ESP8266 là bo Wi-Fi rẻ, phổ biến và có hệ sinh thái thư viện Arduino phong phú cho RC522 và DHT11. Việc tách thành hai bo riêng xuất phát từ giới hạn số chân: màn hình, servo, còi và hai cảm biến cổng đã dùng 10 trong số khoảng 13 chân khả dụng của ESP32-C3 SuperMini.

2.3. Cảm biến và cơ cấu chấp hành

Các cảm biến và cơ cấu chấp hành được chọn theo ba tiêu chí: giá thành thấp, làm việc ở mức logic 3,3 V và tốn ít chân. Bảng 2.2 tổng hợp thông số chính và vai trò của từng linh kiện trong hệ thống.

Bảng 2.2: Cảm biến, cơ cấu chấp hành và thông số chính

Linh kiện	Thông số chính	Vai trò
Cảm biến hồng ngoại phản xạ $\times 7$	Tầm 2–30 cm, bộ so sánh LM393, ngõ ra số tích cực mức thấp	5 ô đỗ và 2 cổng
Cảm biến lửa	Bước sóng 760–1100 nm, góc khoảng 60°, ngõ ra số có ngưỡng	Phát hiện cháy
Cảm biến mực nước	Vùng đo khoảng 40 mm, ngõ ra tương tự	Phát hiện ngập
Đầu đọc RFID MFRC522 $\times 2$	13,56 MHz, ISO 14443A, giao tiếp SPI, 3,3 V	Đọc thẻ ở hai cổng
Cảm biến DHT11	0–50 °C, sai số ± 2 °C, giao thức một dây	Nhiệt độ môi trường
Servo SG90 $\times 2$	4,8–6 V, mô-men khoảng 1,8 kg·cm, điều khiển PWM 50 Hz	Nâng hạ thanh chắn
Màn hình TFT ST7789	1,54 inch, 240×240, IPS, SPI tối đa khoảng 62 MHz	Hiển thị tại cổng
Còi báo (module)	Tích cực mức thấp	Báo cháy, ngập, xe không thẻ

Theo Bảng 2.2, cảm biến hồng ngoại phản xạ được dùng cho cả ô đỗ và cổng vì khoảng cách tới thân xe mô hình chỉ vài centimet, nằm trong tầm đo, và mỗi cảm biến chỉ tốn một chân. Cảm biến siêu âm cho khoảng cách chính xác hơn nhưng tốn hai chân và phải đo thời gian phản hồi. Cảm biến lửa hồng ngoại phản ứng nhanh hơn cảm biến khói hay nhiệt, phù hợp với yêu cầu mở lối thoát kịp thời. Đầu đọc MFRC522 đọc được các loại thẻ MIFARE phổ biến như thẻ sinh viên hay móc khóa từ.

2.4. Giao tiếp ngoại vi

Mỗi thiết bị ngoại vi được nối với vi điều khiển qua một giao tiếp phù hợp với bản chất tín hiệu của nó. Bảng 2.3 liệt kê các giao tiếp được sử dụng.

Bảng 2.3: Các giao tiếp ngoại vi được sử dụng

Giao tiếp	Nguyên lý	Thiết bị
GPIO	Chân vào ra số mức 0 hoặc 1, có điện trở kéo lên hoặc kéo xuống nội	7 cảm biến IR, cảm biến lửa, còi
PWM (LEDC)	Khối phần cứng phát xung có chu kỳ cố định và độ rộng xung điều chỉnh được, CPU không phải đảo chân	2 servo SG90
SPI	Bus nối tiếp đồng bộ gồm SCK, MOSI, MISO và một chân CS cho mỗi thiết bị; có thể dùng DMA	Màn hình ST7789, 2 đầu đọc MFRC522
ADC	Chuyển điện áp tương tự thành số, ESP8266 có một kênh 10 bit	Cảm biến mực nước
Một dây	Giao thức riêng của DHT11, dữ liệu 40 bit mã hóa bằng độ rộng xung	Cảm biến nhiệt độ

Điểm chung của các lựa chọn trong Bảng 2.3 là ưu tiên khối ngoại vi phần cứng thay vì điều khiển chân bằng phần mềm. PWM phần cứng giữ xung servo ổn định ngay cả khi CPU bận vẽ màn hình, còn SPI có DMA truyền 115 KB dữ liệu ảnh mà CPU không phải sao chép từng byte. Nếu tạo xung bằng phần mềm, chỉ một ngắt Wi-Fi cũng đủ làm servo rung.

2.5. Giao thức mạng và thanh toán

Các bo mạch và điện thoại của người vận hành liên lạc qua một mạng Wi-Fi chung. Hệ thống sử dụng các giao thức sau:

- Wi-Fi 802.11 b/g/n băng tần 2,4 GHz ở chế độ trạm, địa chỉ IP do router cấp qua DHCP;
- UDP, giao thức không kết nối, độ trễ thấp, cho phép gửi quảng bá (broadcast) tới mọi thiết bị trong mạng nên bo thu thập không cần biết địa chỉ của máy chủ;
- HTTP cho trang web quản lý và các API dạng JSON, HTTPS khi máy chủ truy vấn SePay;

– mDNS để phân giải tên `parking.local` trong mạng cục bộ mà không cần máy chủ DNS;

– VietQR, chuẩn mã QR chuyển khoản liên ngân hàng của NAPAS dựa trên chuẩn EMVCo, gồm chuỗi các trường dạng mã, độ dài và giá trị, kết thúc bằng mã kiểm tra CRC16.

UDP được chọn thay cho TCP hay MQTT vì phần lớn dữ liệu là trạng thái được gửi lặp lại mỗi giây: mất một gói chỉ làm trễ tối đa một giây, trong khi chi phí bắt tay, bộ đệm cho mỗi kết nối và máy chủ trung gian (broker) là đáng kể trên vi điều khiển. Riêng sự kiện quẹt thẻ không được gửi lặp lại, nên hệ thống bổ sung cơ chế xác nhận và gửi lại ở tầng ứng dụng (Mục 4.2).

2.6. Công cụ phát triển

Toàn bộ phần mềm sử dụng trong đề tài là miễn phí và mã nguồn mở. Bảng 2.4 liệt kê các thành phần chính.

Bảng 2.4: Thành phần phần mềm và vai trò

Thành phần	Vai trò
ESP-IDF 6.1 và FreeRTOS	Framework cho máy chủ: driver, Wi-Fi, TCP/IP, HTTP, NVS, hệ điều hành
`esp_lcd`, LEDC, `esp_timer`	Driver màn hình qua SPI DMA, PWM cho servo, hẹn giờ đóng thanh chắn
`espressif/mdns`, `espressif/qrcode`	Tên `parking.local`; mã hóa mã QR thanh toán
Arduino core ESP8266, thư viện MFRC522 và DHT	Nền tảng và thư viện thiết bị cho hai bo thu thập
HTML, CSS, JavaScript thuần	Trang web quản lý nhúng trong firmware, không phụ thuộc thư viện ngoài
Trình biên dịch C/C++ trên máy tính	Chạy bộ kiểm thử tự động không cần phần cứng

Như Bảng 2.4 cho thấy, trang web được viết bằng HTML, CSS và JavaScript thuần và nhúng trực tiếp vào firmware, nên vẫn mở được khi router mất kết nối Internet. Bộ kiểm thử biên dịch mã nghiệp vụ của máy chủ và mã nguồn hai bo thu thập trên máy tính cùng

các thư viện giả lập phần cứng, giúp kiểm tra logic trong vài giây mà không cần mô hình (Chương 5).

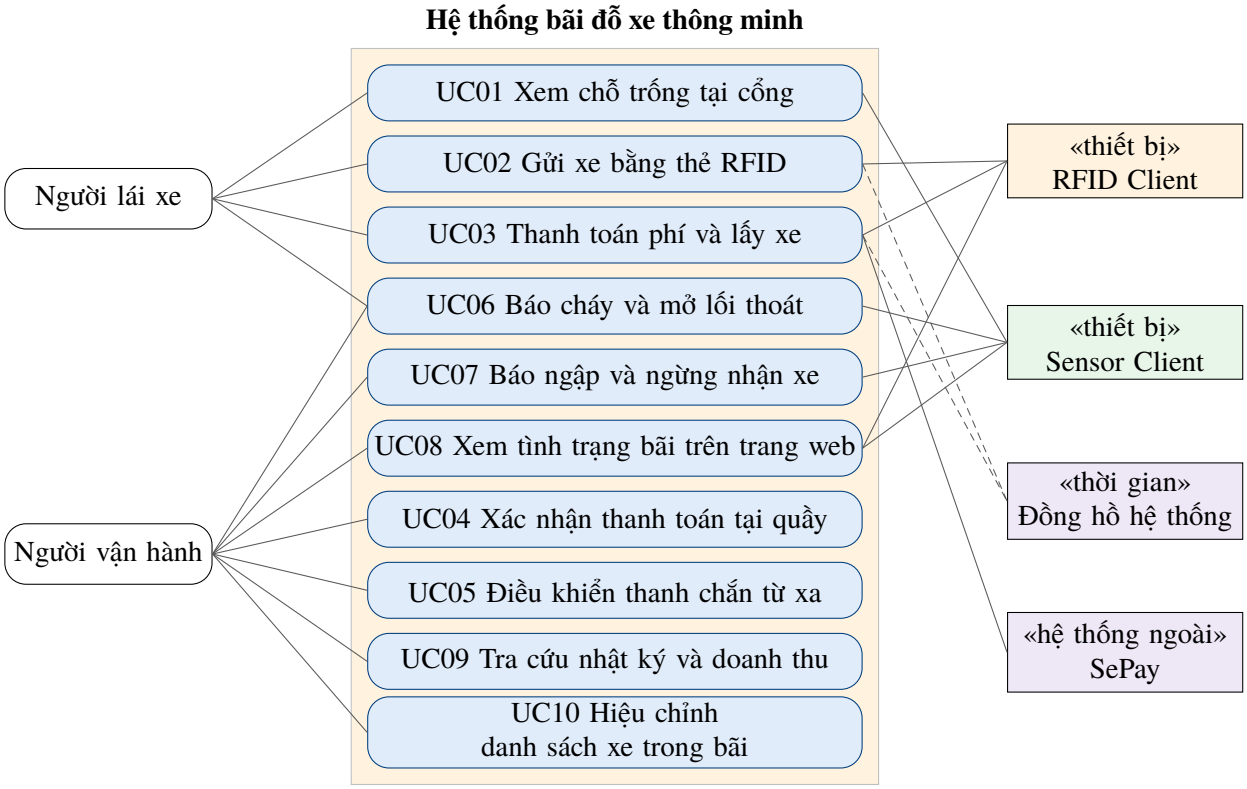
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này đi từ yêu cầu tới thiết kế. Yêu cầu chức năng được mô hình hóa bằng 10 use case, yêu cầu phi chức năng được lượng hóa thành các chỉ tiêu cụ thể. Phần thiết kế trình bày kiến trúc tổng thể, bố trí phần cứng trên mô hình, thuyết minh ba mạch điện, kiến trúc phần mềm của máy chủ và giao thức truyền thông giữa các bo mạch.

3.1. Yêu cầu chức năng

Yêu cầu chức năng được mô hình hóa bằng use case. Mỗi use case biểu diễn một mục tiêu có ý nghĩa của một tác nhân và cho ra một kết quả quan sát được; các tình huống phụ như quẹt sai cổng hay mất gói tin được xem là luồng ngoại lệ của use case chứa nó, không tách thành use case riêng. Theo nguyên tắc này, hệ thống có 10 use case không chồng lấn nhau.

Hệ thống có hai tác nhân chính và bốn tác nhân phụ. Người lái xe là tác nhân chính của các use case gửi xe, lấy xe và xem chỗ trống. Người vận hành là tác nhân chính của các use case quản lý bãi thực hiện qua trang web. Sensor Client và RFID Client là hai tác nhân thiết bị cung cấp dữ liệu cảm biến và sự kiện quẹt thẻ. Đồng hồ hệ thống là tác nhân thời gian kích hoạt việc đóng thanh chắn và hết hạn chờ thanh toán. SePay là hệ thống bên ngoài cung cấp danh sách giao dịch chuyển khoản vào tài khoản của bãi.



Hình 3.1: Sơ đồ use case tổng quát

Hình 3.1 thể hiện quan hệ giữa các tác nhân và 10 use case. Ba use case đầu phục vụ người lái xe trong một lượt gửi xe trọn vẹn; hai use case tiếp theo là phản ứng tự động của hệ thống trước sự cố, có cả người lái và người vận hành tham gia; năm use case còn lại là nghiệp vụ của người vận hành trên trang web. Đường nét đứt biểu diễn sự kiện theo thời gian như đóng thanh chắn hay hết hạn chờ thanh toán. Bảng 3.1 tóm tắt tác nhân chính và mục tiêu của từng use case; đặc tả đầy đủ và cách hiện thực được trình bày theo đúng thứ tự này ở Chương 4.

Bảng 3.1: Tổng hợp các use case

Mã	Tên use case	Tác nhân chính	Mục tiêu
UC01	Xem chỗ trống tại cổng	Người lái xe	Biết bãi còn bao nhiêu chỗ và chỗ nào trống trước khi vào
UC02	Gửi xe bằng thẻ RFID	Người lái xe	Được mở cổng vào và bắt đầu tính thời gian gửi xe
UC03	Thanh toán phí và lấy xe	Người lái xe	Trả đúng phí bằng VietQR và được mở cổng ra
UC04	Xác nhận thanh toán tại quầy	Người vận hành	Cho xe ra khi khách trả tiền mặt hoặc không đối soát tự động được
UC05	Điều khiển thanh chắn từ xa	Người vận hành	Mở hoặc đóng từng cổng hay cả hai cổng khi cần
UC06	Báo cháy và mở lối thoát	Sensor Client	Báo động và giải phóng lối ra ngay khi có lửa
UC07	Báo ngập và ngừng nhận xe	Sensor Client	Báo động và không cho thêm xe vào khi nước dâng
UC08	Xem tình trạng bãi trên trang web	Người vận hành	Nắm được trạng thái ô đỗ, môi trường và thanh chắn từ xa
UC09	Tra cứu nhật ký và doanh thu	Người vận hành	Đối soát lượt xe vào ra và tiền thu được
UC10	Hiệu chỉnh danh sách xe trong bãi	Người vận hành	Xóa xe đã rời bãi mà không qua quy trình ra bình thường

3.2. Yêu cầu phi chức năng

Bên cạnh chức năng, hệ thống phải đáp ứng các yêu cầu về thời gian, an toàn, độ tin cậy và tài nguyên. Bảng 3.2 lượng hóa từng yêu cầu và nêu giải pháp thiết kế tương ứng.

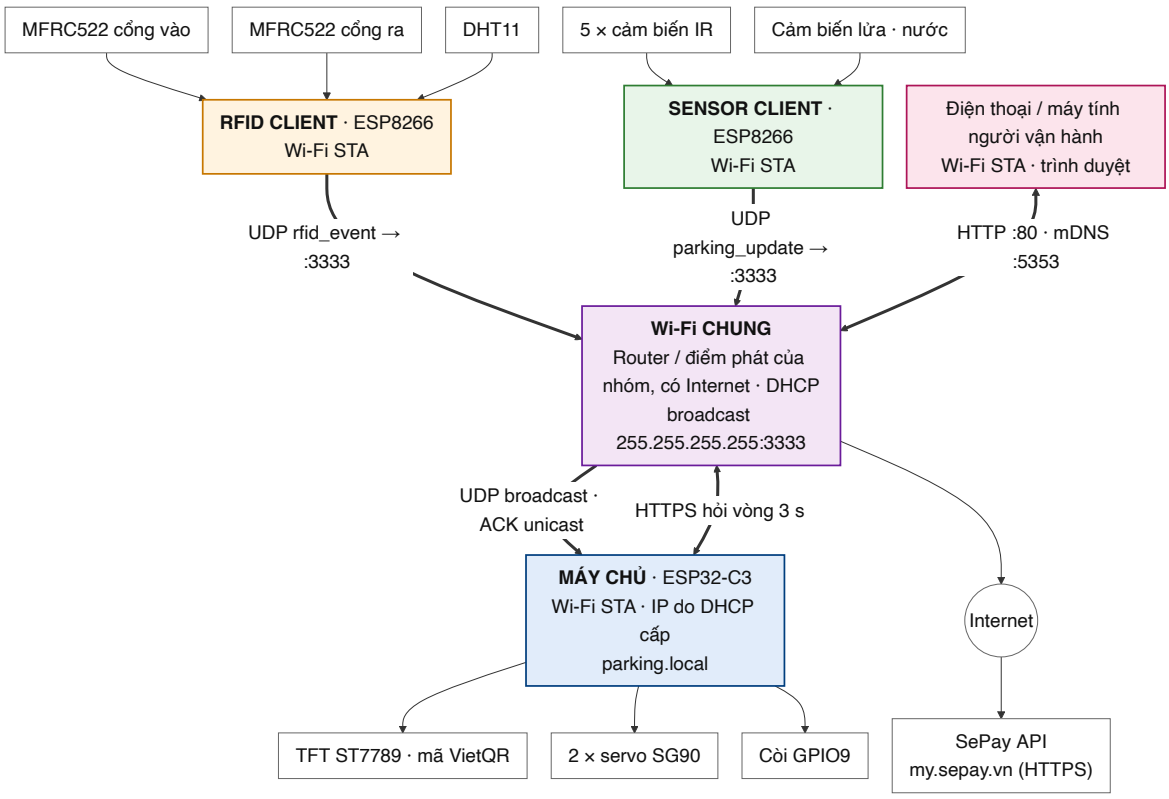
Bảng 3.2: Yêu cầu phi chức năng và giải pháp đáp ứng

Yêu cầu	Chỉ tiêu	Giải pháp thiết kế
Thời gian đáp ứng	Màn hình cập nhật dưới 0,2 s sau khi xe vào ô; thanh chắn mở dưới 0,5 s sau khi quẹt thẻ	Gửi dữ liệu khi có thay đổi, vẽ màn hình theo sự kiện, tác vụ mạng có ưu tiên cao
An toàn	Có cháy thì mở cả hai cổng ngay; không hạ thanh chắn khi xe còn trong cổng; có ngập thì không nhận xe	Chế độ cháy ưu tiên cao nhất; cảm biến IR tại cổng; chặn quẹt thẻ vào khi ngập
Độ tin cậy	Không mất lượt quẹt thẻ khi mất gói; không mất dữ liệu gửi xe khi mất điện	Xác nhận và gửi lại; lưu phiên vào bộ nhớ flash
Tài nguyên	Chạy trong khoảng 321 KB RAM cùng ảnh màn hình 115 KB và kết nối TLS	Cấp phát bộ đệm TLS động, giải phóng bộ đệm ảnh động khi thanh toán
Bảo mật	Thao tác điều khiển cần xác thực; thông tin bí mật không nằm trong mã nguồn	Mã PIN quản trị; cấu hình bí mật qua menuconfig
Dễ sử dụng	Dùng được trên mọi điện thoại, thanh toán bằng ứng dụng ngân hàng bất kỳ	Trang web tại `parking.local`, mã VietQR

Các chỉ tiêu trong Bảng 3.2 chi phối nhiều quyết định thiết kế ở các mục sau. Yêu cầu thời gian dẫn tới việc gửi dữ liệu theo sự kiện thay vì theo chu kỳ; yêu cầu an toàn dẫn tới việc bổ sung cảm biến hồng ngoại dưới mỗi thanh chắn; yêu cầu tài nguyên buộc máy chủ phải quản lý bộ nhớ động một cách có chủ đích khi thực hiện kết nối HTTPS.

3.3. Kiến trúc tổng thể

Hệ thống gồm ba bo mạch cùng kết nối vào một mạng Wi-Fi chung có Internet. Hình 3.2 mô tả kiến trúc triển khai và luồng dữ liệu giữa các thành phần.



Hình 3.2: Kiến trúc triển khai và luồng dữ liệu

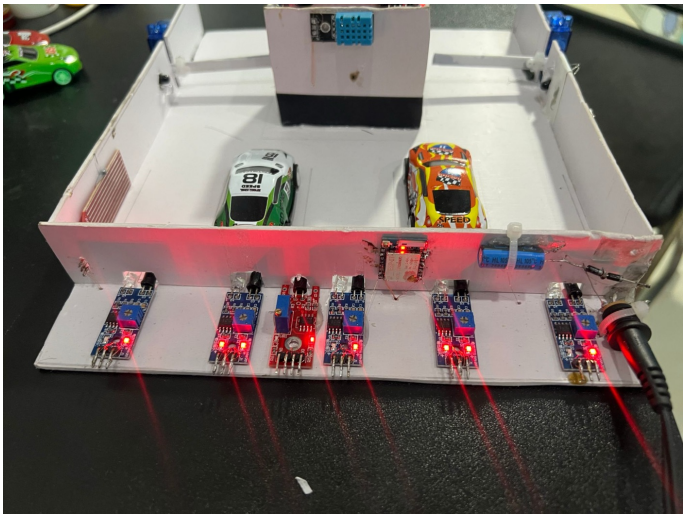
Theo Hình 3.2, mạng có cấu trúc hình sao quanh router; cả ba bo và điện thoại của người vận hành đều là trạm Wi-Fi, nên thứ tự bật nguồn không quan trọng. Hai bo thu thập gửi gói UDP tới địa chỉ quảng bá `255.255.255.255`, cổng 3333, nên không cần biết dải mạng hay địa chỉ IP của máy chủ. Người vận hành truy cập máy chủ bằng tên cố định `parking.local`; máy chủ chủ động thông báo lại tên này mỗi khi được cấp địa chỉ mới, nên tên luôn trỏ đúng thiết bị dù DHCP thay đổi địa chỉ. Router đồng thời là đường ra Internet để máy chủ truy vấn giao dịch trên SePay. Yêu cầu duy nhất đối với router là phát sóng 2,4 GHz, vì cả ESP32-C3 và ESP8266 đều không có sóng 5 GHz, và tắt chế độ cô lập thiết bị để các gói quảng bá được chuyển tiếp.

Kiến trúc tách thành ba bo mang lại ba lợi ích: đủ chân cho toàn bộ thiết bị, đặt cảm biến gần nơi cần đo để dây ngắn, và cô lập lỗi khi một bo mất điện. Đổi lại, hệ thống phụ thuộc vào mạng Wi-Fi; Mục 4.8 trình bày cách máy chủ phát hiện bo thu thập mất kết nối.

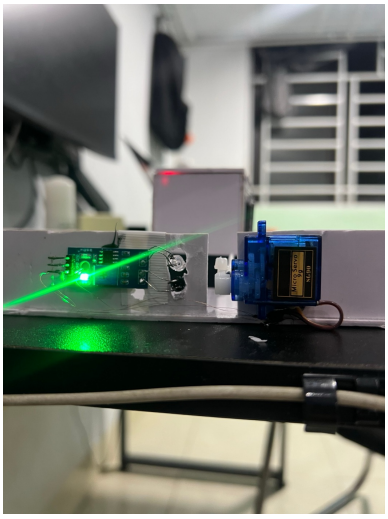
3.4. Thiết kế phần cứng

3.4.1. Bố trí thiết bị trên mô hình

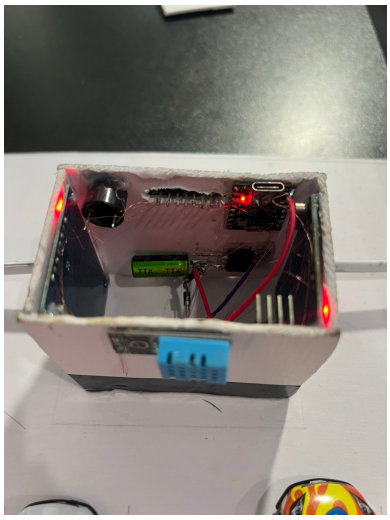
Mô hình được làm bằng bìa formex trắng, vật liệu nhẹ, cách điện và dễ khoét lỗ cho cảm biến. Hình 3.3 cho thấy ba khu vực lắp thiết bị chính ngoài hộp điều khiển đã giới thiệu ở Hình 1.1.



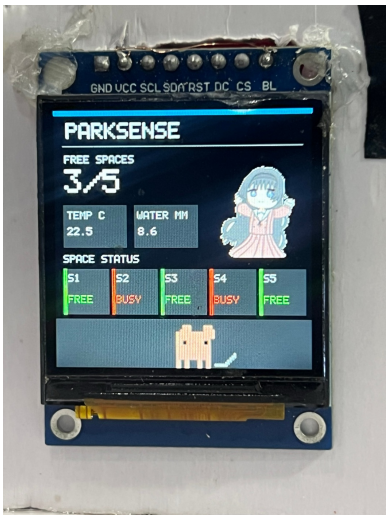
(a) Mặt ngoài vách sau: Sensor Client với 5 module IR, module lửa, mạch nguồn



(b) Cổng ra: servo SG90 và module IR phát hiện xe trong cổng



(c) Bên trong hộp điều khiển: ESP32-C3, còi, mạch nguồn, hai đầu đọc RC522 ở hai thành hộp



(d) Màn hình TFT ở mặt trước hộp điều khiển

Hình 3.3: Bố trí phần cứng trên mô hình

Hình 3.3a là mặt ngoài vách sau, nơi đặt toàn bộ Sensor Client: năm module hồng ngoại có đầu phát và thu xuyên qua lỗ khoét, hướng vào giữa từng ô; module lửa màu đỏ nằm giữa dãy để góc nhìn khoảng 60° bao quát cả năm ô; module ESP8266, tụ lọc, hai diode và jack nguồn nằm cùng mặt vách. Bố trí bo mạch và dây ở mặt ngoài giữ cho lòng bãi gọn và mỗi module vẫn có biến trở để chỉnh ngưỡng theo độ phản xạ thực tế. Cảm biến mực nước đặt sát sàn ở góc trong của vách sau, nơi nước đọng trước tiên. Hình 3.3b là cổng ra: module hồng ngoại gắn ngay trước trục servo, tia hồng ngoại chiếu ngang làn xe dưới cần chắn. Hình 3.3c là bên trong hộp điều khiển, nơi đặt máy chủ ESP32-C3, còi, RFID Client cùng mạch nguồn của nó; hai đầu đọc RC522 gắn ở hai thành hộp, mỗi đầu đọc quay ra một làn, và cảm biến DHT11 gắn ở mặt ngoài hộp để không đo nhầm nhiệt tỏa ra từ bo mạch. Hình 3.3d là màn hình ở mặt trước hộp, quay về phía người lái.

3.4.2. Sơ đồ chân

Việc gán chân tuân theo hai nguyên tắc: tránh các chân UART dùng để nạp chương trình, và chỉ nối vào chân strapping những tín hiệu có mức nghỉ phù hợp với chế độ khởi động bình thường. Bảng 3.3 tổng hợp sơ đồ chân của cả ba bo.

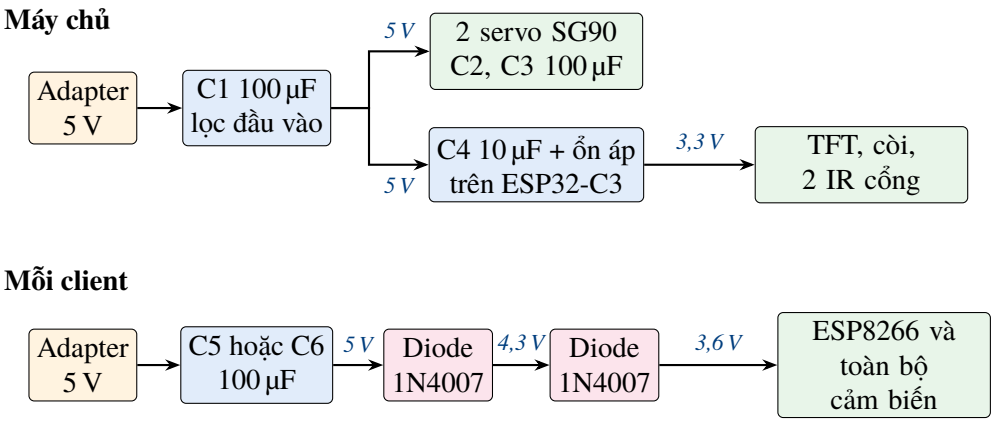
Bảng 3.3: Sơ đồ chân của ba bo mạch

Tín hiệu	Chân	Ghi chú
Máy chủ ESP32-C3		
TFT SCK, MOSI, RES, DC, CS	GPIO4, 6, 3, 7, 10	SPI2 qua ma trận GPIO, màn hình không cần MISO
Servo cổng vào, cổng ra	GPIO0, 1	PWM LEDC kênh 0 và 1
Còi	GPIO9	Tích cực mức thấp, mức nghỉ cao trùng yêu cầu strapping
IR cổng vào, cổng ra	GPIO5, 2	Tích cực mức thấp, có điện trở kéo lên nội
Đèn nền TFT (BL)	nối 3,3 V	Luôn sáng, nhường GPIO5 cho cảm biến cổng
Sensor Client ESP8266		
IR ô S1 đến S5	GPIO16, 4, 14, 13, 12	Tránh chân strapping 0, 2, 15 và UART 1, 3
Cảm biến lửa, mực nước	GPIO5, A0	A0 là kênh ADC duy nhất của ESP8266
RFID Client ESP8266		
SPI SCK, MOSI, MISO	GPIO14, 13, 12	Dùng chung cho hai đầu đọc
SS cổng vào, cổng ra; RST	GPIO5, 4; GPIO16	Chân chọn chip riêng, chân reset chung
DHT11	GPIO2	Module có điện trở kéo lên, mức nghỉ cao

Bảng 3.3 cho thấy máy chủ đã dùng 10 chân, trong đó GPIO5 vốn điều khiển đèn nền màn hình được chuyển cho cảm biến cổng vào; đèn nền được nối thẳng lên nguồn 3,3 V và luôn sáng. Trên RFID Client, hai đầu đọc dùng chung ba dây SPI và chân reset, chỉ khác nhau ở chân chọn chip, giúp tiết kiệm hai chân so với hai bus riêng.

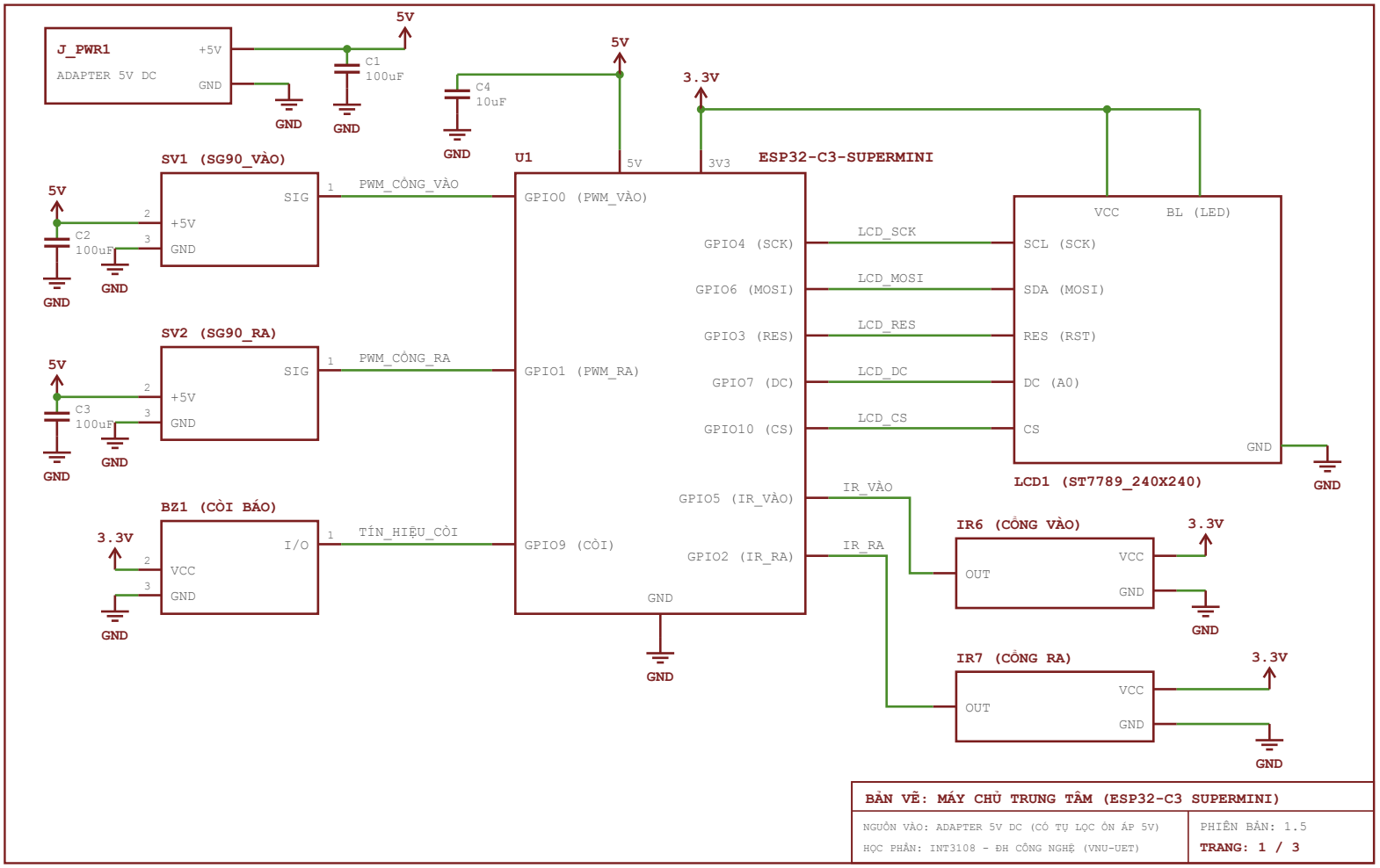
3.4.3. Thuyết minh mạch điện

Mạch điện của hệ thống gồm ba bản vẽ, mỗi bản vẽ ứng với một bo mạch và có đầu vào nguồn 5 V riêng. Trước khi đi vào từng bản vẽ, Hình 3.4 tóm tắt đường đi của nguồn điện, phần quyết định độ ổn định của toàn hệ thống.

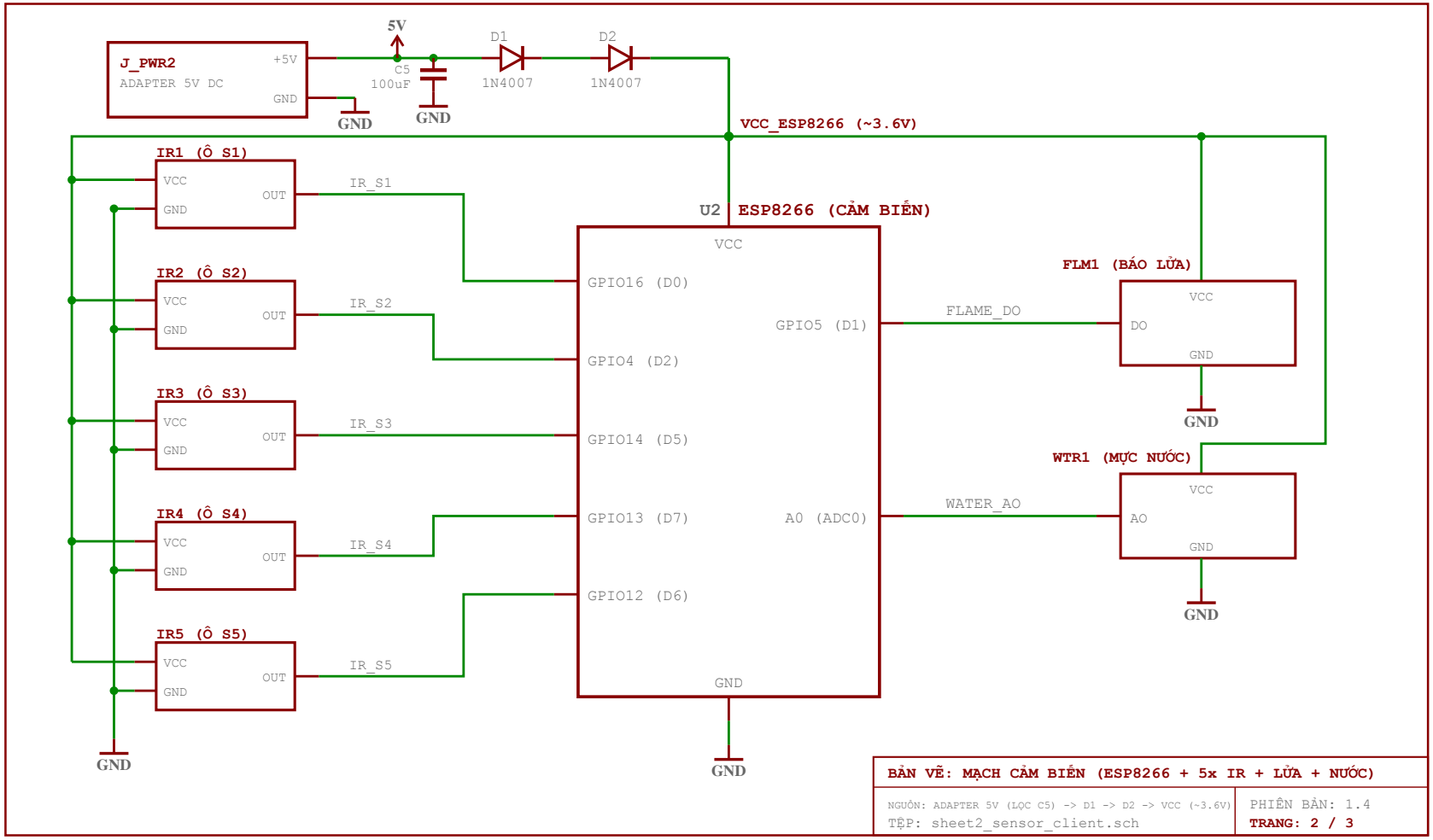


Hình 3.4: Đường cấp nguồn của máy chủ và của mỗi client

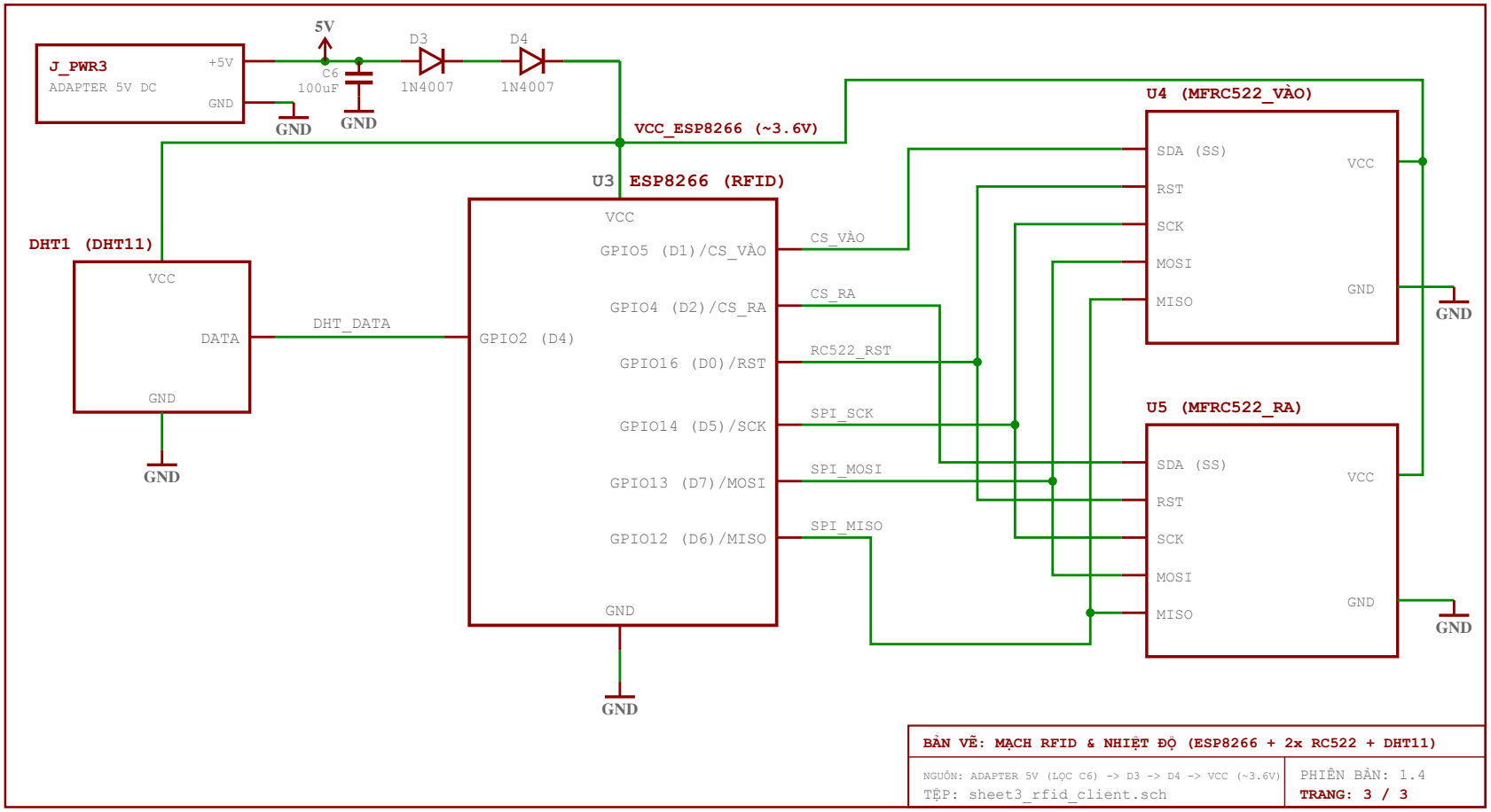
Hình 3.4 cho thấy hai cách cấp nguồn khác nhau. Máy chủ có hai nhánh: nhánh 5 V cấp trực tiếp cho servo, nhánh còn lại đi qua bộ ổn áp 3,3 V có sẵn trên bo ESP32-C3 SuperMini để cấp cho màn hình, còi và hai cảm biến cổng. Mỗi client hạ 5 V xuống khoảng 3,6 V bằng hai diode mắc nối tiếp rồi cấp chung cho ESP8266 và các cảm biến. Ba bản vẽ chi tiết ở Hình 3.5, 3.6 và 3.7 được thuyết minh ngay sau đây.



Hình 3.5: Sơ đồ nguyên lý máy chủ: ESP32-C3, màn hình ST7789, 2 servo, còi và 2 cảm biến IR ở cổng



Hình 3.6: Sơ đồ nguyên lý Sensor Client: ESP8266, 5 cảm biến IR ô đỗ, cảm biến lửa và mực nước



Hình 3.7: Sơ đồ nguyên lý RFID Client: ESP8266, 2 đầu đọc RC522 dùng chung SPI và DHT11

3.4.3.1. Mạch máy chủ

Hình 3.5 là mạch máy chủ. Adapter J_PWR1 cấp đường 5 V chung; tụ hóa C1 100 µF đặt ngay đầu vào san phẳng gợn sóng của adapter và đóng vai trò kho năng lượng cho toàn mạch. Hai servo SG90 lấy nguồn trực tiếp từ đường 5 V thay vì từ chân 3V3 của bo, vì SG90 cần 4,8–6 V và kéo dòng vài trăm mA lúc bắt đầu quay, vượt khả năng của bộ ổn áp nhỏ trên bo. Mỗi servo có một tụ 100 µF riêng (C2, C3) đặt sát chân nguồn. Khi động cơ khởi động, dòng tức thời tăng vọt; nếu không có tụ, điện áp đường 5 V sụt xuống và có thể làm ESP32-C3 tự khởi động lại. Tụ cấp phần dòng tức thời này tại chỗ, đặc biệt cần thiết khi cả hai servo quay cùng lúc trong chế độ cháy. Chân GND của adapter phải nối chung với GND của ESP32-C3 để xung PWM trên GPIO0 và GPIO1 có cùng mức tham chiếu với servo.

Tụ C4 10 µF đặt sát chân 5V của ESP32-C3 lọc nhiễu tần số cao và giữ ổn định điện áp vào bộ ổn áp 3,3 V trên bo trong các đợt phát Wi-Fi công suất cao. Đầu ra 3V3 của bo cấp cho màn hình, còi và hai cảm biến hồng ngoại ở cổng. Cấp 3,3 V cho cảm biến cổng là yêu cầu bắt buộc: ESP32-C3 không chịu được mức 5 V ở chân vào, nên ngõ ra của module phải nằm trong dải 0–3,3 V. Hai cảm biến này tích cực mức thấp, nối vào GPIO5 (cổng vào) và GPIO2 (cổng ra) với điện trở kéo lên nội được bật trong firmware.

Còi được nối vào GPIO9 và hai cảm biến cổng dùng GPIO5, GPIO2; trong đó GPIO9 và GPIO2 là chân strapping được đọc lúc khởi động để chọn chế độ, và mức cao ở cả hai chân ứng với chế độ chạy chương trình bình thường. Module còi tích cực mức thấp nên mức nghỉ là mức cao, trùng với yêu cầu này; firmware cũng đặt GPIO9 ở mức cao ngay khi khởi tạo. Cảm biến cổng ra có mức nghỉ cao khi làn trống, vì vậy không được che tia hồng ngoại cổng ra trong lúc cấp nguồn hoặc nạp chương trình. Màn hình ST7789 nối qua SPI2 với năm tín hiệu SCK, MOSI, RES, DC và CS; chân đèn nền BL nối thẳng lên 3,3 V.

3.4.3.2. Mạch Sensor Client

Hình 3.6 là mạch Sensor Client. Adapter J_PWR2 cấp 5 V qua tụ C5 100 µF, sau đó qua hai diode chỉnh lưu 1N4007 (D1, D2) mắc nối tiếp để tạo đường VCC khoảng 3,6 V cấp

chung cho module ESP8266 và các cảm biến. Mỗi diode silic có điện áp rơi thuận khoảng 0,7 V khi dẫn, nên điện áp sau hai diode là

$$V_{CC} \approx V_{in} - 2V_F = 5 - 2 \times 0{,}7 = 3{,}6\ \text{V}, \tag{3.1}$$

nằm trong dải 3,0–3,6 V của ESP8266. Cặp diode đảm nhiệm hai chức năng cùng lúc. Chức năng thứ nhất là hạ áp mà không cần IC ổn áp, phù hợp với linh kiện sẵn có và dòng tiêu thụ nhỏ của client. Chức năng thứ hai là bảo vệ ngược cực: nếu adapter bị cắm ngược, diode phân cực ngược và chặn dòng, bảo vệ ESP8266 cùng các module. Công suất tiêu tán trên mỗi diode là $P = V_F \cdot I \approx 0{,}7 \times 0{,}3 \approx 0{,}2$ W ở đỉnh dòng khi phát Wi-Fi, thấp hơn nhiều so với định mức 1 A của 1N4007. Hạn chế của phương án là V_F thay đổi theo dòng tải, nên điện áp VCC dao động quanh 3,6 V; tụ C5 phía trước giữ ổn định đầu vào, còn trong sản phẩm thực tế nên thay bằng bộ ổn áp LDO 3,3 V.

Năm cảm biến hồng ngoại IR1 đến IR5 cùng lấy nguồn từ đường VCC, ngõ ra OUT nối lần lượt vào GPIO16, 4, 14, 13 và 12. Vì cảm biến dùng chung nguồn với ESP8266, mức cao ở ngõ ra không vượt quá điện áp chịu đựng của chân vào. Cảm biến lửa FLM1 nối ngõ ra số DO vào GPIO5, cảm biến mực nước WTR1 nối ngõ ra tương tự AO vào chân ADC A0.

3.4.3.3. Mạch RFID Client

Hình 3.7 là mạch RFID Client, có đường nguồn giống Sensor Client gồm adapter J_PWR3, tụ C6 100 µF và hai diode D3, D4. Tụ C6 có thêm vai trò ổn định điện áp khi hai ăng-ten RC522 đóng cắt trường từ 13,56 MHz và khi ESP8266 phát gói tin. Hai đầu đọc U4 (cổng vào) và U5 (cổng ra) dùng chung bus SPI gồm SCK, MOSI, MISO ở GPIO14, 13, 12 và chân reset ở GPIO16; mỗi đầu đọc có chân chọn chip SDA riêng là GPIO5 và GPIO4, nên tại mỗi thời điểm chỉ một đầu đọc giao tiếp trên bus. Cảm biến DHT11 nối chân DATA vào GPIO2; đây là chân strapping của ESP8266, cần mức cao khi khởi động, và điện trở kéo lên sẵn có trên module DHT11 bảo đảm điều kiện này.

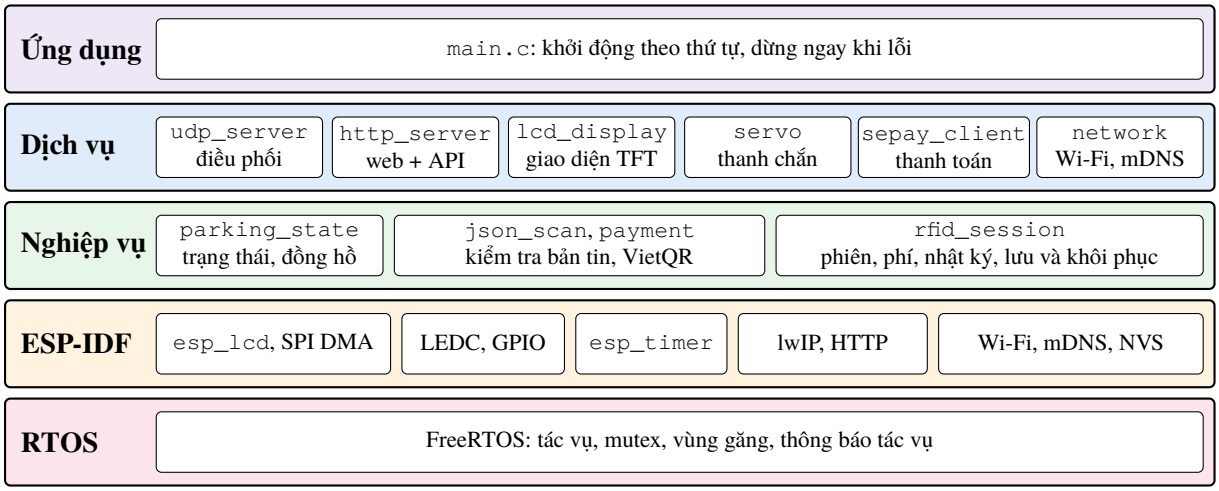
Bảng 3.4 tổng hợp chức năng của các linh kiện thụ động và linh kiện nguồn xuất hiện trong ba bản vẽ, làm cơ sở đối chiếu khi lắp ráp hoặc sửa chữa.

Bảng 3.4: Chức năng của các linh kiện nguồn và linh kiện thụ động

Ký hiệu	Giá trị	Chức năng
J_PWR1, 2, 3	Adapter 5 V	Nguồn cho máy chủ, Sensor Client, RFID Client
C1	100 µF	Lọc gợn sóng và dự trữ năng lượng ở đầu vào máy chủ
C2, C3	100 µF	Cấp dòng khởi động cho servo cổng vào, cổng ra, chống sụt áp
C4	10 µF	Lọc nhiễu cao tần tại chân 5V của ESP32-C3
C5, C6	100 µF	Lọc nguồn 5 V trước cặp diode của mỗi client
D1, D2 và D3, D4	1N4007	Hạ áp 5 V xuống khoảng 3,6 V và bảo vệ ngược cực

3.5. Kiến trúc phần mềm máy chủ

Firmware máy chủ được tổ chức theo kiến trúc phân lớp như Hình 3.8, trong đó mỗi lớp chỉ gọi lớp nằm dưới nó.



Hình 3.8: Kiến trúc phân lớp của firmware máy chủ

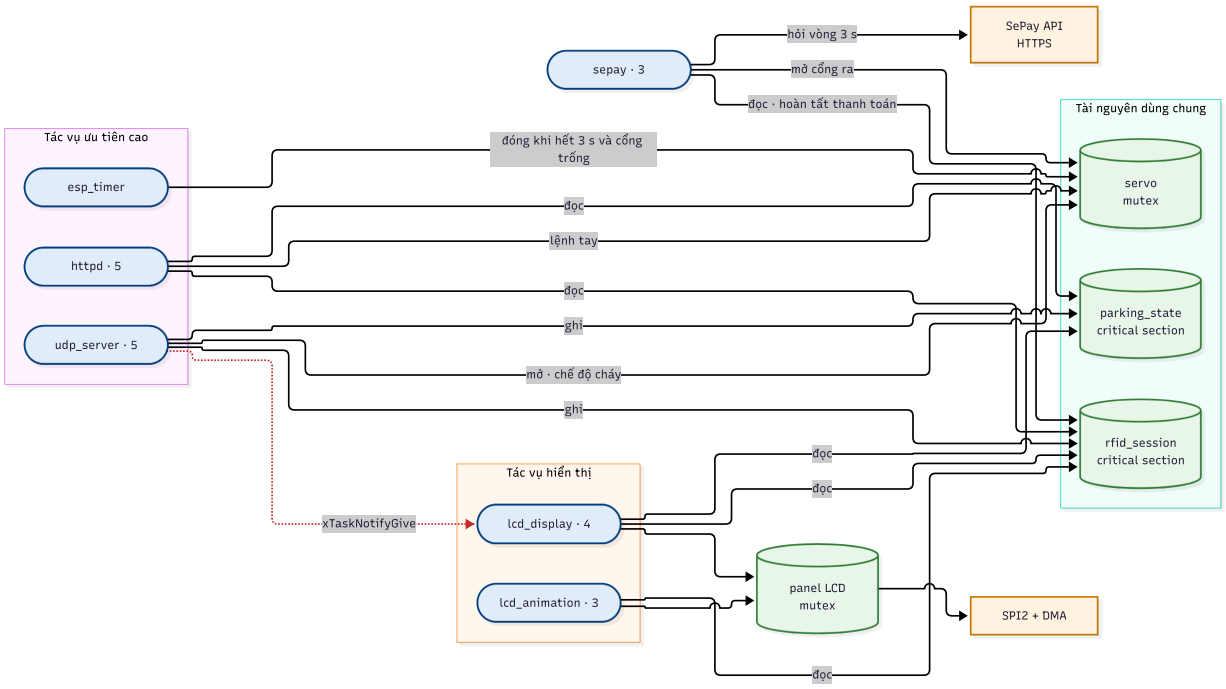
Theo Hình 3.8, lớp nghiệp vụ gồm trạng thái bãi, kiểm tra bản tin, phiên gửi xe và thanh toán không gọi trực tiếp phần cứng hay mạng, chỉ cần hai macro vùng găng và một hàm đọc đồng hồ. Thay hai phần này bằng phiên bản giả, toàn bộ lớp nghiệp vụ được biên dịch và kiểm thử trên máy tính. Lớp dịch vụ chứa các module điều khiển thiết bị và mạng; trong đó `udp_server` đóng vai trò điều phối, nhận bản tin, gọi lớp nghiệp vụ rồi ra lệnh cho servo, còi và màn hình, nhờ vậy các module thiết bị không phụ thuộc lẫn nhau. Đổi màn hình khác chỉ cần sửa `lcd_display`, đổi giao thức mạng chỉ cần sửa `udp_server`.

Các module trên chạy trong bảy tác vụ FreeRTOS với mức ưu tiên khác nhau, được liệt kê ở Bảng 3.5.

Bảng 3.5: Các tác vụ chạy trên máy chủ

Tác vụ	Ưu tiên	Được đánh thức bởi	Nhiệm vụ
`udp_server`	5	Gói UDP đến	Kiểm tra bản tin, cập nhật trạng thái, ra lệnh thiết bị
`httpd`	5	Yêu cầu HTTP	Trang web và API
`esp_timer`	cao	Hẹn giờ	Kiểm tra cảm biến cổng và đóng thanh chắn
`lcd_display`	4	Thông báo tác vụ, chu kỳ 1 s	Vẽ lại màn hình khi dữ liệu đổi, nhịp còi cảnh báo
`lcd_animation`	3	Mỗi 100 ms	Ảnh động trang trí trên màn hình chính
`sepay`	3	Mỗi 3 s khi có xe đang thanh toán	Truy vấn giao dịch qua HTTPS
`mdns`	1	Gói mDNS	Trả lời truy vấn `parking.local`

Mức ưu tiên trong Bảng 3.5 phản ánh mức độ quan trọng đối với người dùng: nhận bản tin và điều khiển thanh chắn liên quan trực tiếp tới xe đang chờ ở cổng nên có ưu tiên cao nhất; màn hình thông tin quan trọng hơn ảnh động trang trí; tác vụ SePay chờ mạng hàng trăm mili giây mỗi lần nên không được chen trước tác vụ điều khiển. Khi CPU bận, ảnh động chậm lại còn thông tin vẫn kịp thời. Hình 3.9 cho thấy các tác vụ cùng truy cập những tài nguyên dùng chung nào và cơ chế bảo vệ tương ứng.



Hình 3.9: Tác vụ, tài nguyên dùng chung và cơ chế đồng bộ

Hình 3.9 thể hiện ba cơ chế đồng bộ được dùng cho ba loại tình huống. Trạng thái bãi và bảng phiên gửi xe được bảo vệ bằng vùng găng, vì đoạn cần bảo vệ chỉ là vài lệnh sao chép cấu trúc nhỏ, rẻ hơn mutex. Servo và màn hình được bảo vệ bằng mutex, vì thao tác ghi thanh ghi LEDC hay truyền SPI kéo dài hơn và mutex có cơ chế kế thừa ưu tiên chống đảo ưu tiên. Đường nét đứt màu đỏ là thông báo tác vụ: `udp_server` đánh thức `lcd_display` ngay khi có dữ liệu mới, thay cho việc hỏi vòng định kỳ.

3.6. Thiết kế giao thức truyền thông

Hai bo thu thập gửi ba loại bản tin JSON tới máy chủ qua UDP. Mã nguồn 3.1 là ví dụ của từng loại cùng gói xác nhận mà máy chủ gửi lại.

```
1 {"type":"parking_update","slots":[1,0,0,1,0],"water_mm":0.2,"fire":
    false}
2 {"type":"climate","temperature_c":28.0}
3 {"type":"rfid_event","card_id":"A1B2C3D4","gate":"in","temperature_c":
    28.0,"seq":305441741}
4 {"type":"rfid_ack","seq":305441741}
```

Mã nguồn 3.1: Các bản tin UDP giữa client và máy chủ

Trong Mã nguồn 3.1, bản tin `parking_update` mang toàn bộ trạng thái của Sensor Client gồm năm ô đỗ, mực nước và cờ cháy; bản tin `climate` mang nhiệt độ; bản tin `rfid_event` mang mã thẻ, cổng và số thứ tự `seq`. Vì cổng UDP là ranh giới tin cậy mà mọi thiết bị trong mạng đều gửi tới được, máy chủ kiểm tra chặt từng bản tin bằng bộ đọc JSON tự viết, không cấp phát bộ nhớ động và chỉ chấp nhận đúng lược đồ: mảng `slots` đúng 5 phần tử 0 hoặc 1, mực nước 0–10 000 mm, nhiệt độ -40 đến 125 °C, mã thẻ 1–31 ký tự chữ, số, gạch nối; khóa lạ, khóa lặp hay dữ liệu thừa đều bị từ chối và trạng thái cũ được giữ nguyên. So với thư viện cJSON, cách làm này tránh phân mảnh bộ nhớ khi chạy lâu và giới hạn được thời gian xử lý mỗi bản tin.

Phía trang web, máy chủ cung cấp các API HTTP được liệt kê ở Bảng 3.6. Mọi lệnh làm thay đổi trạng thái dùng phương thức POST và phải mang mã PIN quản trị trong header `X-Parking-Key`; sai mã PIN, máy chủ trả mã lỗi 401.

Bảng 3.6: API HTTP của máy chủ

Phương thức	Đường dẫn	Chức năng
GET	`/api/parking`	Trạng thái ô đỗ, số chỗ trống, nhiệt độ, mực nước, cháy, ngập
GET	`/api/barrier`	Trạng thái hai thanh chắn
GET	`/api/rfid`	Xe trong bãi, lượt thanh toán đang chờ, 10 sự kiện gần nhất, doanh thu
POST	`/api/barrier/<cổng>/<lệnh>`	Mở hoặc đóng cổng `in`, `out` hoặc `all`
POST	`/api/payment/confirm`	Người vận hành xác nhận đã nhận tiền
POST	`/api/rfid/remove`	Xóa một xe khỏi danh sách trong bãi
POST	`/api/time`	Trình duyệt cung cấp giờ thực cho máy chủ

Các API trong Bảng 3.6 được trang web gọi theo chu kỳ 1–2 s. Cách hỏi vòng này không giữ trạng thái kết nối trên máy chủ, mỗi yêu cầu được xử lý xong là giải phóng, phù hợp với giới hạn bộ nhớ và số kết nối đồng thời của vi điều khiển; độ trễ thêm tối đa một giây là chấp nhận được với dữ liệu thay đổi theo giây.

CHƯƠNG 4. HIỆN THỰC HỆ THỐNG THEO USE CASE

Chương này trình bày quá trình hiện thực theo đúng thứ tự 10 use case ở Bảng 3.1. Mỗi mục mở đầu bằng bảng đặc tả use case gồm tác nhân, mục tiêu, tiền điều kiện, sự kiện kích hoạt, luồng chính, luồng ngoại lệ và hậu điều kiện; phần tiếp theo phân tích cách hiện thực trên vi điều khiển và lý do chọn các thông số. Kết quả kiểm thử tương ứng được tổng hợp ở Chương 5.

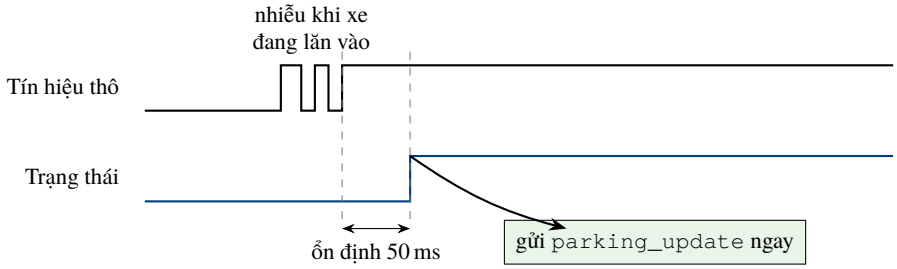
4.1. UC01. Xem chỗ trống tại cổng

Use case này giúp người lái quyết định có vào bãi hay không và cần đi tới ô nào. Đặc tả của use case được trình bày ở Bảng 4.1.

Bảng 4.1: Đặc tả UC01. Xem chỗ trống tại cổng

Mục	Nội dung
Tác nhân chính	Người lái xe
Tác nhân phụ	Sensor Client
Mục tiêu	Biết số chỗ trống và vị trí từng chỗ trống trước khi vào bãi
Tiền điều kiện	Máy chủ và Sensor Client đã kết nối vào mạng Wi-Fi
Kích hoạt	Một xe đi vào hoặc rời khỏi một ô đỗ
Luồng chính	– Cảm biến hồng ngoại của ô đổi mức logic. – Sensor Client lọc nhiễu 50 ms rồi gửi ngay bản tin `parking_update`. – Máy chủ kiểm tra bản tin, cập nhật trạng thái bãi và đánh thức tác vụ màn hình. – Màn hình tại cổng hiển thị số chỗ trống và năm ô S1 đến S5, vạch xanh cho ô trống, vạch đỏ cho ô có xe. – Người lái đọc thông tin trên màn hình.
Luồng ngoại lệ	– Gói tin bị mất: bản tin nhắc lại sau tối đa 1 s cập nhật lại trạng thái. – Bản tin sai định dạng: máy chủ bỏ qua và giữ trạng thái cũ. – Sensor Client im lặng quá 3 s: màn hình hiển thị `SENSOR OFFLINE`, năm ô chuyển xám.
Hậu điều kiện	Màn hình phản ánh đúng trạng thái năm ô đỗ với độ trễ khoảng 0,1 s

Bước lọc nhiễu trong luồng chính là cần thiết vì mép xe và bánh xe làm tín hiệu hồng ngoại bật tắt liên tục trong vài chục mili giây khi xe đang lăn vào ô. Hình 4.1 minh họa thuật toán: mỗi cảm biến có bộ đếm thời gian riêng, mức mới chỉ được chấp nhận khi giữ nguyên đủ 50 ms.



Hình 4.1: Lọc nhiễu tín hiệu cảm biến ô đỗ

Thời gian 50 ms được tinh chỉnh thực nghiệm từ giá trị ban đầu 250 ms: đủ loại bỏ nhiễu mà vẫn tạo cảm giác tức thời. Ngoài việc gửi ngay khi có thay đổi, Sensor Client còn nhắc lại toàn bộ trạng thái mỗi giây; nhờ vậy, một gói UDP bị mất chỉ gây sai lệch tối đa một giây và tự được sửa ở lần gửi sau, không cần cơ chế truyền lại.

Ở phía máy chủ, tác vụ `udp_server` đánh thức tác vụ màn hình bằng thông báo tác vụ ngay khi có dữ liệu mới. Màn hình dùng một bộ đệm khung 240×240 điểm ảnh 16 bit (115 200 byte): toàn bộ giao diện được vẽ vào RAM rồi truyền một lần qua SPI 40 MHz có DMA, mất khoảng 23 ms. Tần số 40 MHz bằng đúng một nửa xung nhịp 80 MHz của khối SPI và thấp hơn giới hạn khoảng 62 MHz của ST7789, chừa biên an toàn cho dây nối. Giao diện thực tế ở Hình 3.3d gồm hàng trên là số chỗ trống, ô nhiệt độ và ô mực nước kèm một ảnh động trang trí; hàng giữa là năm ô đỗ; vùng chân màn hình dành cho hóa đơn và thông báo. Khi Sensor Client mất kết nối, dữ liệu cũ không còn được dùng để từ chối thẻ hay phát cảnh báo, tránh việc hệ thống hành động dựa trên trạng thái không còn đúng.

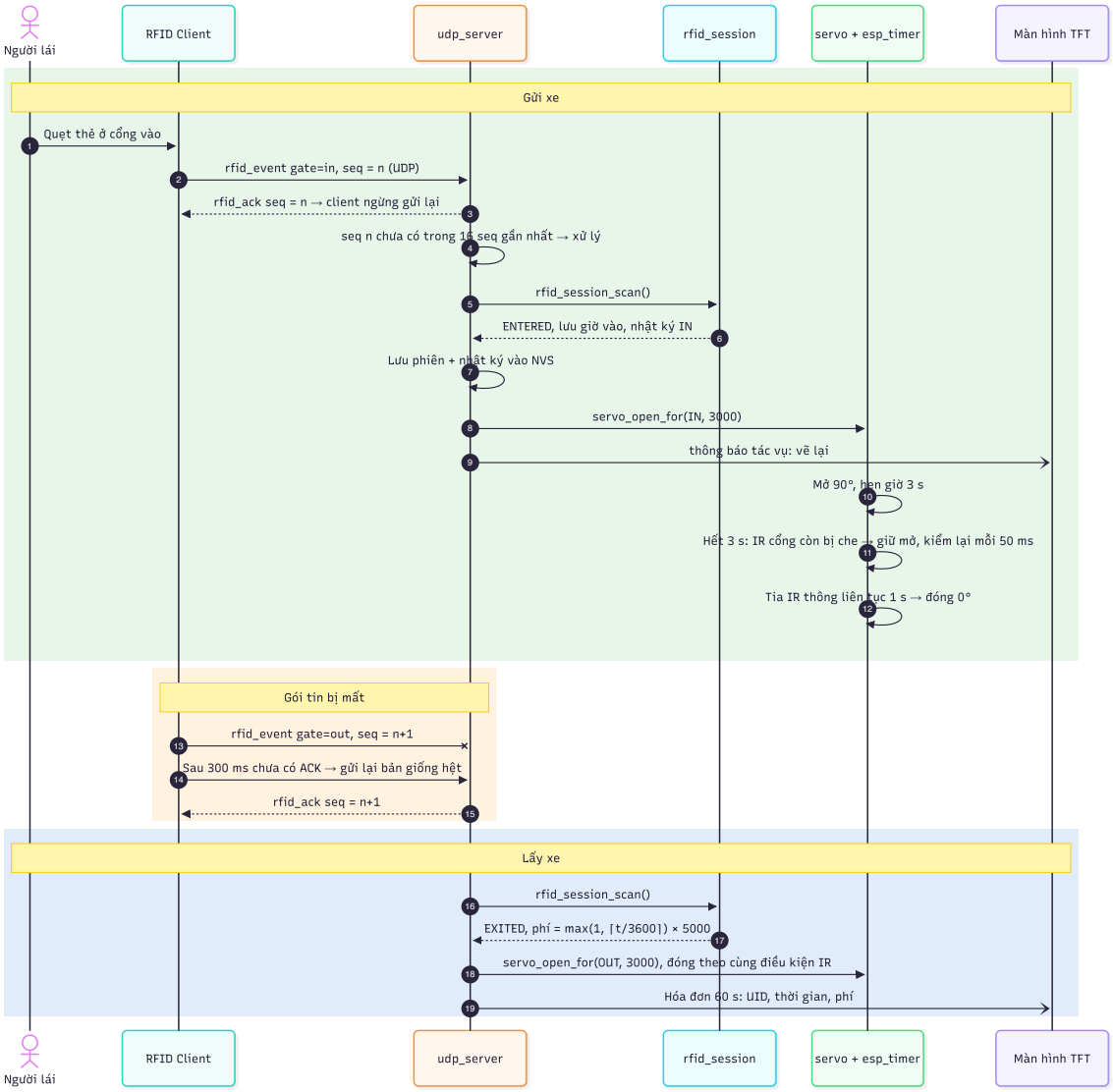
4.2. UC02. Gửi xe bằng thẻ RFID

Use case này mô tả một lượt xe vào bãi, bao gồm cả các trường hợp hệ thống phải từ chối. Đặc tả được trình bày ở Bảng 4.2.

Bảng 4.2: Đặc tả UC02. Gửi xe bằng thẻ RFID

Mục	Nội dung
Tác nhân chính	Người lái xe
Tác nhân phụ	RFID Client, Đồng hồ hệ thống
Mục tiêu	Được mở cổng vào và bắt đầu tính thời gian gửi xe
Tiền điều kiện	Thẻ chưa có phiên gửi xe; bãi còn chỗ; không có cảnh báo ngập
Kích hoạt	Người lái đưa thẻ vào đầu đọc ở cổng vào
Luồng chính	– RFID Client đọc mã UID và gửi `rfid_event` với `"gate":"in"` kèm số thứ tự `seq`. – Máy chủ trả gói xác nhận, tạo phiên mới, lưu thời điểm vào, ghi nhật ký `IN` và lưu bảng phiên vào flash. – Máy chủ mở thanh chắn cổng vào và hẹn giờ 3 s. – Xe đi qua cổng; hết 3 s, thanh chắn hạ xuống khi cảm biến hồng ngoại ở cổng xác nhận làn trống.
Luồng ngoại lệ	– Thẻ đã có phiên trong bãi: không mở cổng, màn hình báo `WRONG GATE` và gợi ý dùng cổng ra trong 5 s. – Bãi đã đủ 5 phiên hoặc cảm biến báo đủ 5 ô có xe: không mở cổng, màn hình báo `PARKING FULL`. – Đang có cảnh báo ngập: không mở cổng, màn hình báo `FLOOD ALARM – ENTRY CLOSED`. – Gói tin bị mất: sau 300 ms chưa có xác nhận, RFID Client gửi lại, tối đa 6 lần; máy chủ nhận ra bản gửi lại qua `seq` nên không mở cổng lần hai. – Xe còn nằm dưới cần chắn khi hết 3 s: thanh chắn giữ mở cho tới khi làn trống liên tục 1 s. – Một xe khác đi theo mà không quẹt thẻ: sau 15 s số ô có xe vẫn nhiều hơn số thẻ, còi kêu ngắt quãng và màn hình báo `CAR W/O CARD`.
Hậu điều kiện	Thẻ có phiên gửi xe, nhật ký có bản ghi `IN`, thanh chắn đóng

Hình 4.2 biểu diễn trình tự trao đổi giữa các thành phần trong luồng chính, trường hợp gói tin bị mất và luồng ra khi không bật thanh toán QR.



Hình 4.2: Biểu đồ tuần tự gửi xe, xử lý mất gói và lấy xe

4.2.1. Đọc thẻ trên hai đầu đọc dùng chung bus

RFID Client quét luân phiên hai đầu đọc và tại mỗi thời điểm chỉ bật ăng-ten của một đầu đọc. Hai đầu đọc cùng tần số 13,56 MHz đặt gần nhau sẽ gây nhiễu chéo nếu cùng phát, nên cách bật luân phiên loại bỏ hoàn toàn vấn đề này. Sau khi bật ăng-ten, client chờ 4 ms để thẻ thụ động nhận đủ năng lượng; độ khuếch đại được đặt ở mức 48 dB để bù tầm đọc khi đầu đọc nằm sau thành hộp. Một thẻ để yên trên đầu đọc sẽ bị đọc liên tục, vì vậy client bỏ qua các lần đọc lặp lại cùng một thẻ trong vòng 2 s, tránh trường hợp một lần quẹt bị

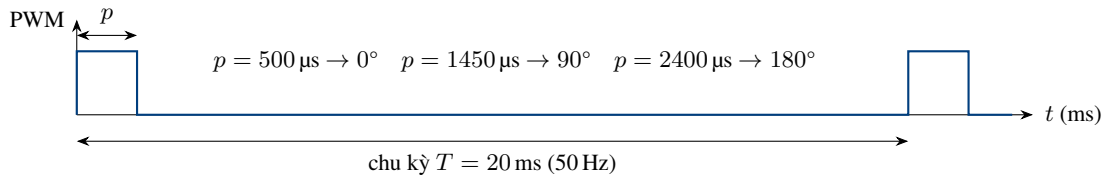
hiểu thành vào rồi ra. Mã UID được đổi sang chuỗi hex trong bộ đệm cố định thay vì kiểu `String` của Arduino, để bộ nhớ heap khoảng 80 KB của ESP8266 không bị phân mảnh khi chạy lâu.

4.2.2. Bảo đảm không mất lượt quẹt thẻ

Khác với trạng thái cảm biến, sự kiện quẹt thẻ chỉ xảy ra một lần; mất gói đồng nghĩa với mất lượt quẹt. Thay vì chuyển sang TCP, hệ thống bổ sung đúng phần độ tin cậy cần thiết như ở phần giữa của Hình 4.2. Máy chủ ghi nhớ 16 giá trị `seq` gần nhất; gặp lại một `seq` đã xử lý, máy chủ hiểu đó là bản gửi lại, chỉ trả xác nhận mà không xử lý lần hai. Giá trị `seq` khởi đầu từ một số ngẫu nhiên do bộ sinh số ngẫu nhiên phần cứng của ESP8266 tạo ra mỗi lần khởi động rồi tăng dần, nên client không cần lưu gì vào flash.

4.2.3. Điều khiển thanh chắn

Servo SG90 nhận xung chu kỳ 20 ms, độ rộng xung quyết định góc quay như Hình 4.3.



Hình 4.3: Dạng xung điều khiển servo SG90

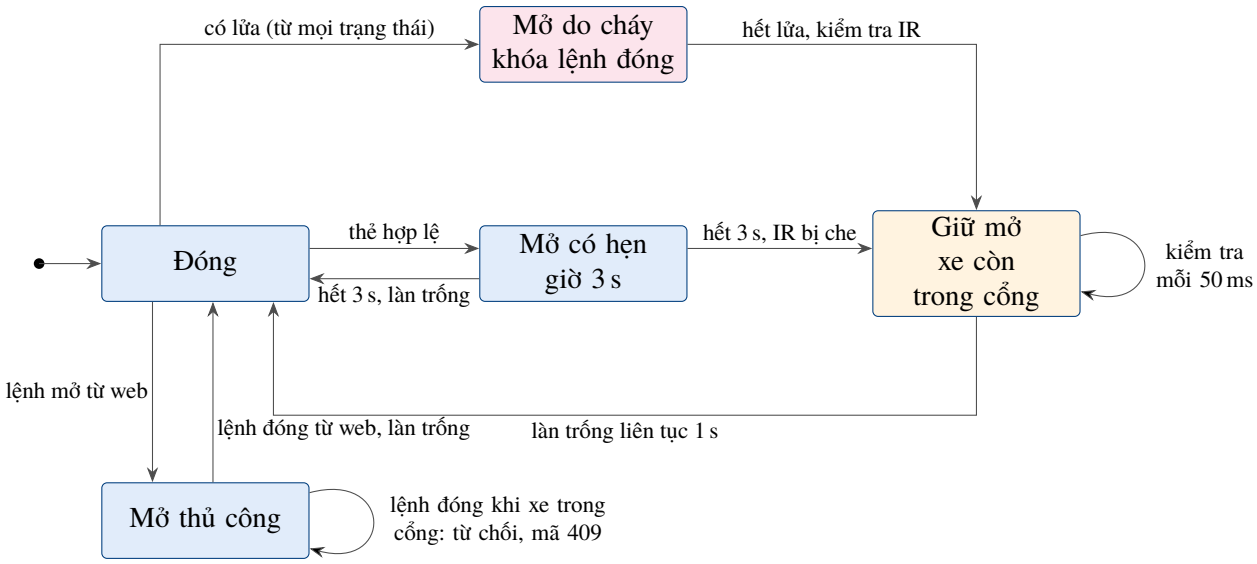
Từ Hình 4.3, độ rộng xung p ứng với góc θ và giá trị D nạp vào thanh ghi của khối LEDC 14 bit được tính theo

$$p(\theta) = 500 + \frac{1900\,\theta}{180}\ \mu s, \qquad D = \left\lfloor \frac{p \cdot (2^{14} - 1)}{20\,000} \right\rfloor . \tag{4.1}$$

Với $\theta = 90°$ ta có $p = 1450\,\mu s$ và $D = 1187$; với $\theta = 0°$ ta có $p = 500\,\mu s$ và $D = 409$. Độ phân giải 14 bit cho mỗi bước khoảng 1,22 µs, tương đương 0,12°, mịn hơn nhiều so với vùng chết khoảng 1° của SG90 nên servo không rung. Dải 500–2400 µs là giá trị đo thực tế; dải danh nghĩa 1000–2000 µs không đủ để servo quay hết hành trình. Thanh chắn cổng ra được lắp

đối xứng nên góc mở và đóng đảo ngược so với cổng vào, khác biệt này chỉ được khai báo ở một hàm.

Việc đóng thanh chắn được hẹn bằng bộ hẹn giờ một lần `esp_timer` thay vì một tác vụ hỏi vòng: hàm đóng được gọi đúng thời điểm với sai số micro giây và CPU không bị đánh thức vô ích. Hình 4.4 là máy trạng thái của một thanh chắn, dùng chung cho UC02, UC03, UC05 và UC06.

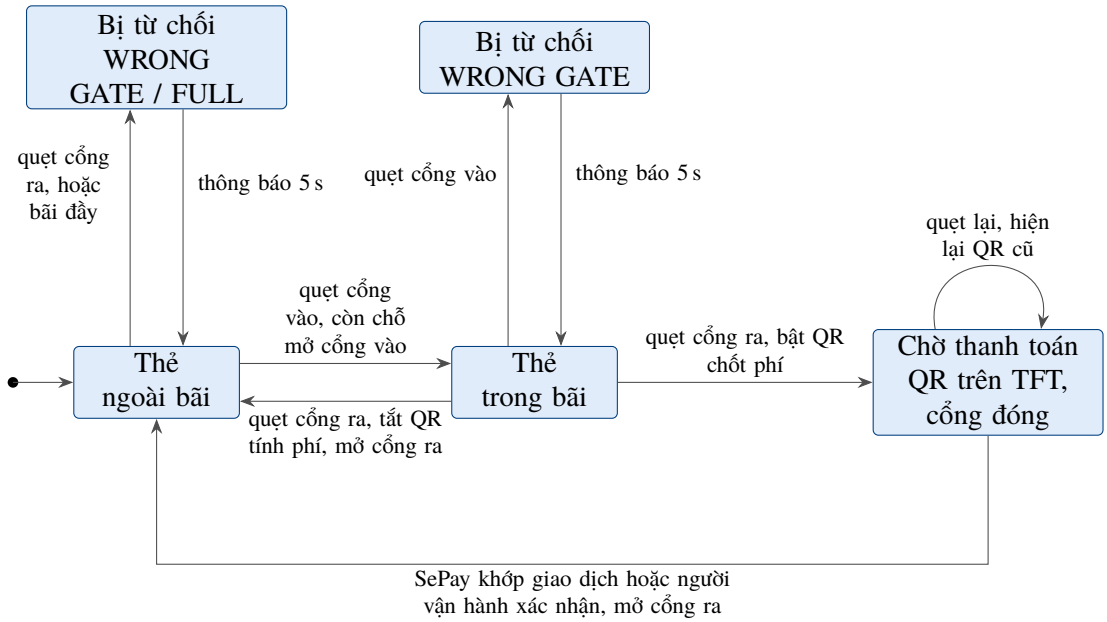


Hình 4.4: Máy trạng thái của một thanh chắn

Theo Hình 4.4, khi hết 3 s, hàm đóng đọc cảm biến hồng ngoại của cổng. Nếu tia bị che, tức là xe còn nằm dưới cần chắn, thanh chắn được giữ mở và hàm đóng tự hẹn lại sau 50 ms để kiểm tra tiếp. Thanh chắn chỉ hạ xuống khi tia thông suốt liên tục 1 s; điều kiện liên tục này loại bỏ trường hợp cảm biến nháy một mẫu đúng lúc khoảng trống giữa hai trục xe đi qua. Logic đếm thời gian thông suốt được tách thành một hàm thuần không phụ thuộc phần cứng để kiểm thử trên máy tính. Vì một lệnh khác có thể xảy ra đồng thời với hàm đóng, hàm đóng kiểm tra lại hạn đóng trong vùng bảo vệ bởi mutex. Cả hai thanh chắn luôn khởi động ở vị trí đóng, vì vậy trạng thái thanh chắn không cần lưu vào flash.

4.2.4. Từ chối lượt quẹt không hợp lệ

Các luồng ngoại lệ từ chối của UC02 và UC03 cùng dựa trên trạng thái của thẻ. Hình 4.5 tổng hợp các trạng thái của một thẻ và các chuyển trạng thái, bao gồm cả các nhánh bị từ chối.



Hình 4.5: Máy trạng thái của một thẻ RFID

Hình 4.5 cho thấy hai trạng thái bị từ chối đều tự quay về trạng thái trước đó sau 5 s, nghĩa là lần quẹt sai không làm thay đổi phiên gửi xe; nếu ngay sau đó có một lần quẹt hợp lệ, thông báo cũ bị xóa ngay để không che thông tin mới. Điều kiện bãi đầy được xét theo cả số phiên lẫn số ô có xe theo cảm biến, nên hệ thống vẫn từ chối thẻ mới khi có xe đã vào mà không quẹt thẻ. Trường hợp xe đi theo không quẹt thẻ được phát hiện bằng cách so sánh liên tục hai số này; ngưỡng 15 s đủ dài để một xe vừa quẹt thẻ có thời gian đi vào ô mà không gây báo động nhầm.

4.3. UC03. Thanh toán phí và lấy xe

Use case này mô tả một lượt xe ra bãi khi bật thanh toán VietQR. Đặc tả được trình bày ở Bảng 4.3.

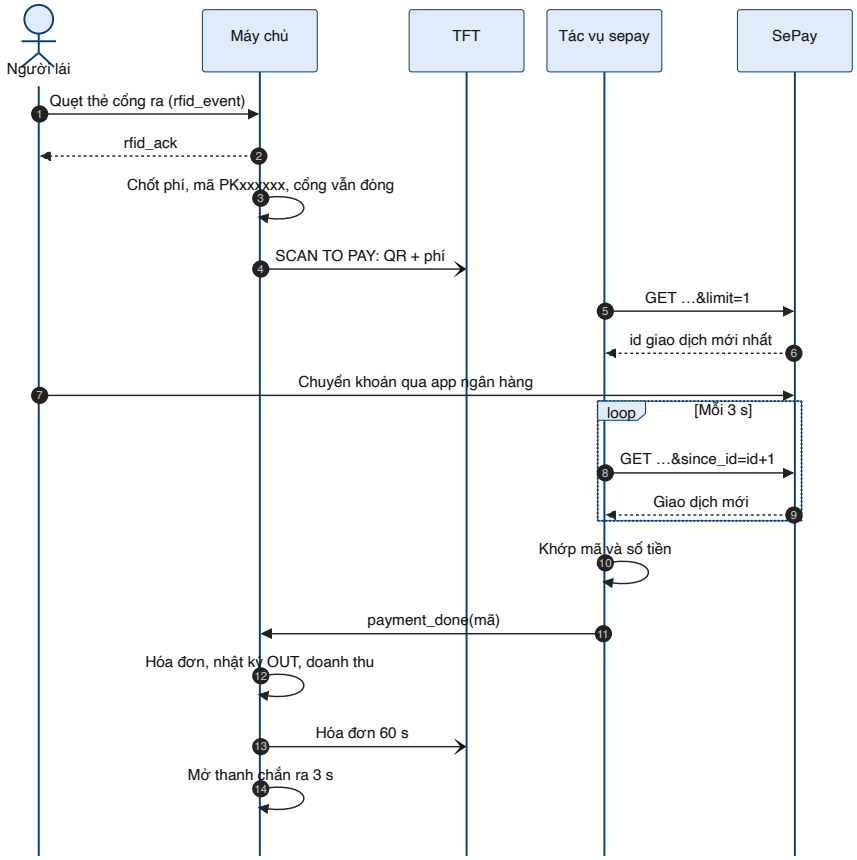
Bảng 4.3: Đặc tả UC03. Thanh toán phí và lấy xe

Mục	Nội dung
Tác nhân chính	Người lái xe
Tác nhân phụ	RFID Client, SePay, Đồng hồ hệ thống
Mục tiêu	Trả đúng phí gửi xe bằng chuyển khoản và được mở cổng ra
Tiền điều kiện	Thẻ đang có phiên gửi xe; tài khoản nhận và khóa truy cập SePay đã được cấu hình
Kích hoạt	Người lái đưa thẻ vào đầu đọc ở cổng ra
Luồng chính	– Máy chủ tính thời gian gửi và phí, chốt phí, tạo mã thanh toán gồm tiền tố `PK` và 6 ký tự. – Thanh chắn vẫn đóng; màn hình hiển thị mã VietQR, số tiền và nội dung chuyển khoản. – Người lái quét mã bằng ứng dụng ngân hàng và chuyển khoản với số tiền đã điền sẵn. – Máy chủ truy vấn SePay mỗi 3 s và tìm thấy giao dịch khớp mã và đúng số tiền. – Máy chủ đóng phiên, ghi nhật ký `OUT`, cộng doanh thu; màn hình hiển thị lời cảm ơn 5 s rồi hóa đơn 60 s. – Thanh chắn cổng ra mở và đóng theo cùng điều kiện cảm biến cổng như UC02.
Luồng ngoại lệ	– Thẻ chưa có phiên gửi xe: không mở cổng, màn hình báo `WRONG GATE` và gợi ý dùng cổng vào. – Người lái quẹt lại cùng thẻ: màn hình hiển thị lại đúng mã QR cũ, thời gian chờ được tính lại. – Xe khác quẹt khi lượt trước còn hiệu lực: mã QR của xe trước được giữ, dòng cuối màn hình báo `NEXT CAR: WAIT, ONE IS PAYING`. – Chuyển dư hoặc thiếu tiền: giao dịch không được chấp nhận, cổng vẫn đóng. – Quá 180 s chưa nhận được tiền: màn hình báo `PAY AT BOOTH` trong 30 s; tiếp tục ở UC04.
Hậu điều kiện	Phiên được giải phóng; nhật ký có bản ghi `OUT`; doanh thu tăng đúng bằng phí

Phí gửi xe được tính theo giờ, làm tròn lên và tối thiểu một giờ:

$$\text{phí} = 5000 \times \max\left(1, \left\lceil \frac{t}{3600} \right\rceil\right) \ \text{VNĐ}, \tag{4.2}$$

trong đó t là thời gian gửi tính bằng giây. Công thức được hiện thực bằng phép chia nguyên `(t + 3599) / 3600` để tránh số thực, và bão hòa ở giá trị lớn nhất của kiểu 32 bit nếu tràn. Mã thanh toán dùng 31 ký tự gồm chữ in hoa và chữ số, đã loại bỏ `I`, `L`, `O`, `0`, `1` vì dễ nhầm khi người lái phải gõ tay. Hình 4.6 là trình tự trao đổi trong quá trình thanh toán.



Hình 4.6: Biểu đồ tuần tự thanh toán VietQR ở cổng ra

4.3.1. Sinh mã VietQR trên vi điều khiển

Chuỗi VietQR được máy chủ tự dựng theo chuẩn của NAPAS, vốn là định dạng EMVCo: mỗi trường gồm mã 2 chữ số, độ dài 2 chữ số và giá trị, các trường 38 và 62 chứa trường con theo cùng quy tắc. Bảng 4.4 liệt kê các trường trong chuỗi do máy chủ sinh ra.

Bảng 4.4: Cấu trúc chuỗi VietQR do máy chủ sinh

Mã	Trường	Giá trị
00	Phiên bản định dạng	`01`
01	Kiểu mã	`12`, mã động dùng cho một số tiền cụ thể
38	Thông tin người nhận	Định danh NAPAS `A000000727`, mã BIN ngân hàng và số tài khoản, dịch vụ `QRIBFTTA`
53, 58	Tiền tệ, quốc gia	`704` (VNĐ), `VN`
54	Số tiền	Phí đã chốt, ví dụ `10000`
62	Dữ liệu bổ sung	Nội dung chuyển khoản là mã thanh toán, ví dụ `PK7XK2QM`
63	Mã kiểm tra	CRC16-CCITT (đa thức `0x1021`, giá trị đầu `0xFFFF`)

Trường 54 trong Bảng 4.4 cố định số tiền đúng bằng phí, và trường 62 cố định nội dung chuyển khoản là mã thanh toán. Hàm sinh chuỗi được kiểm chứng trùng từng byte với một mã VietQR mẫu đã công bố và với giá trị kiểm tra chuẩn `0x29B1` của CRC16-CCITT. Mã QR được vẽ bằng thư viện qrcodegen với mức sửa lỗi M; việc sinh mã hoàn toàn trên chip nên vẫn hoạt động khi mất Internet. Nếu số tài khoản không hợp lệ hoặc để trống, máy chủ không vẽ mã QR mà hiển thị `PAY AT BOOTH` và việc thu tiền chuyển sang UC04.

4.3.2. Đối soát giao dịch qua SePay

SePay theo dõi biến động số dư của tài khoản nhận và cho phép truy vấn giao dịch qua API. Khi một lượt thanh toán bắt đầu, tác vụ `sepay` ghi lại mã giao dịch mới nhất, sau đó cứ 3 s truy vấn các giao dịch có mã lớn hơn, nên chỉ những giao dịch xảy ra sau khi hiển thị mã QR mới được xét. Một giao dịch được chấp nhận khi nội dung chứa mã thanh toán và số tiền đúng bằng phí; chuyển dư hay thiếu đều không mở cổng, vì mã QR chỉ điền sẵn số tiền còn việc khóa ô nhập là do từng ứng dụng ngân hàng quyết định. Thử nghiệm thực tế cho thấy một số ví điện tử như MoMo thay nội dung chuyển khoản bằng nội dung riêng, do đó máy chủ chấp nhận thêm giao dịch vào đúng tài khoản nhận với số tiền đúng bằng phí. Vì mỗi lúc chỉ có một xe thanh toán và chỉ xét giao dịch sau thời điểm hiện mã, cách đối soát này không nhầm lẫn giữa các xe.

Máy chủ dùng phương pháp hỏi vòng thay vì webhook vì ESP32-C3 nằm sau NAT của router, không có địa chỉ để Internet gọi tới. Chu kỳ 3 s cho độ trễ vài giây, thấp hơn nhiều so với giới hạn 2–3 yêu cầu mỗi giây của SePay. Kết nối HTTPS cần thêm bộ nhớ cho TLS

trong khi RAM đã chật bởi bộ đệm khung 115 KB; vì vậy bộ đệm TLS chỉ được cấp phát trong lúc gửi yêu cầu, và tác vụ ảnh động trả lại hai bộ đệm 18 KB của nó trong lúc có xe đang thanh toán, khi ảnh động vốn không hiển thị.

4.4. UC04. Xác nhận thanh toán tại quầy

Thanh toán tự động phụ thuộc vào Internet, dịch vụ SePay và việc người lái chuyển khoản đúng. Khi một trong các điều kiện này không thỏa mãn, hoặc khi người lái trả tiền mặt, bãi vẫn phải cho xe ra. Bảng 4.5 đặc tả use case dành cho tình huống này.

Bảng 4.5: Đặc tả UC04. Xác nhận thanh toán tại quầy

Mục	Nội dung
Tác nhân chính	Người vận hành
Mục tiêu	Cho xe ra khi đã thu tiền bằng cách khác ngoài đối soát tự động
Tiền điều kiện	Thẻ đang có phiên gửi xe; người vận hành biết mã PIN quản trị
Kích hoạt	Người vận hành bấm `Confirm paid` tại xe trong danh sách hoặc tại băng rôn `WAITING FOR PAYMENT`
Luồng chính	– Người vận hành thu tiền mặt hoặc kiểm tra giao dịch trong ứng dụng ngân hàng. – Trang web gửi `POST /api/payment/confirm` kèm mã thẻ và mã PIN. – Máy chủ tính phí tới thời điểm xác nhận nếu xe chưa quẹt ra, hoặc dùng phí đã chốt nếu xe đang chờ thanh toán. – Máy chủ đóng phiên, ghi nhật ký `OUT`, cộng doanh thu, hiển thị hóa đơn và mở cổng ra.
Luồng ngoại lệ	– Sai hoặc thiếu mã PIN: máy chủ trả mã 401, trang web yêu cầu nhập lại. – Thẻ không có phiên gửi xe: máy chủ trả lỗi, không có thay đổi.
Hậu điều kiện	Như hậu điều kiện của UC03

Theo Bảng 4.5, UC04 đi qua đúng đường xử lý ra bãi của UC03 sau bước đối soát, nên hóa đơn, nhật ký và doanh thu nhất quán giữa hai cách thanh toán và một xe không thể ra hai lần. Trang web nhận lượt thanh toán đang chờ qua trường `payment` của `GET /api/rfid` để hiển thị băng rôn màu vàng gồm mã thẻ, phí và mã thanh toán. Việc yêu cầu mã PIN bảo đảm chỉ người có thẩm quyền mới cho được xe ra mà không qua đối soát tự động.

4.5. UC05. Điều khiển thanh chắn từ xa

Use case này cho phép người vận hành chủ động mở hoặc đóng thanh chắn, chẳng hạn khi cần cho xe ưu tiên đi qua hoặc khi có sự cố với đầu đọc thẻ. Đặc tả được trình bày ở Bảng 4.6.

Bảng 4.6: Đặc tả UC05. Điều khiển thanh chắn từ xa

Mục	Nội dung
Tác nhân chính	Người vận hành
Mục tiêu	Đưa một hoặc cả hai thanh chắn về trạng thái mong muốn
Tiền điều kiện	Thiết bị của người vận hành ở cùng mạng Wi-Fi với máy chủ; có mã PIN quản trị
Kích hoạt	Người vận hành bấm `OPEN` hoặc `CLOSE` của một cổng, hoặc nút khẩn cấp cho cả hai cổng
Luồng chính	– Trang web gửi `POST /api/barrier/<cổng>/<lệnh>` kèm mã PIN và khóa nút tới khi có phản hồi. – Máy chủ hủy hẹn giờ tự đóng nếu có và đặt góc servo. – Trang web hiển thị trạng thái mới `OPEN` hoặc `CLOSED`.
Luồng ngoại lệ	– Sai hoặc thiếu mã PIN: máy chủ trả mã 401, trang web hỏi mã PIN. – Lệnh đóng khi xe còn trong cổng: máy chủ trả mã 409, trang web hiển thị `CAR IN GATE`, thanh chắn giữ mở. – Lệnh đóng khi đang có cháy: lệnh bị từ chối, thanh chắn giữ mở.
Hậu điều kiện	Thanh chắn ở trạng thái người vận hành chọn, trừ khi việc đóng gây mất an toàn

Như Bảng 4.6 cho thấy, lệnh thủ công được ưu tiên hơn hẹn giờ tự động nhưng không được vượt qua hai điều kiện an toàn là xe trong cổng và cháy. Mã PIN được lưu trong trình duyệt sau lần nhập đầu tiên và được máy chủ so sánh theo thời gian không đổi, để thời gian phản hồi không làm lộ thông tin về mã PIN. Mã lỗi 409 riêng cho trường hợp xe trong cổng giúp người vận hành hiểu lý do lệnh bị từ chối thay vì nghĩ hệ thống gặp lỗi.

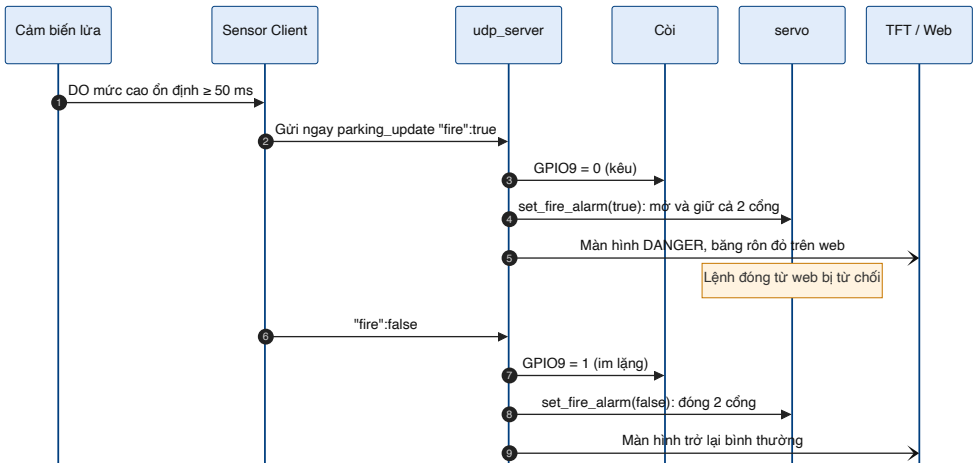
4.6. UC06. Báo cháy và mở lối thoát

Khi có cháy, hệ thống phải báo động và giải phóng lối ra mà không chờ người vận hành. Đặc tả được trình bày ở Bảng 4.7.

Bảng 4.7: Đặc tả UC06. Báo cháy và mở lối thoát

Mục	Nội dung
Tác nhân chính	Sensor Client
Tác nhân phụ	Người lái xe, Người vận hành
Mục tiêu	Báo động tại chỗ và từ xa, giữ hai cổng mở để xe thoát ra
Tiền điều kiện	Sensor Client đang gửi dữ liệu
Kích hoạt	Cảm biến lửa phát hiện ngọn lửa ổn định trong 50 ms
Luồng chính	– Sensor Client gửi ngay bản tin có `"fire":true`. – Máy chủ bật còi, mở và giữ mở cả hai thanh chắn. – Màn hình tại cổng chuyển sang giao diện `DANGER, FIRE DETECTED, EVACUATE NOW`; trang web hiển thị băng rôn đỏ. – Khi hết lửa, máy chủ tắt còi, đưa màn hình về bình thường và đóng hai thanh chắn khi làn trống.
Luồng ngoại lệ	– Người vận hành ra lệnh đóng trong lúc cháy: lệnh bị từ chối. – Sensor Client mất kết nối trong lúc cháy: trạng thái cháy cuối cùng được giữ nguyên.
Hậu điều kiện	Lối ra vào thông thoáng trong suốt thời gian có lửa

Hình 4.7 là trình tự xử lý từ cảm biến lửa tới các thiết bị chấp hành.



Hình 4.7: Biểu đồ tuần tự cảnh báo cháy

Theo Hình 4.7, máy chủ bật còi bằng cách kéo GPIO9 xuống mức thấp và gọi chế độ cháy của module servo; chế độ này có ưu tiên cao nhất trong máy trạng thái ở Hình 4.4. Tổng độ trễ từ lúc có lửa tới lúc còi kêu khoảng 60 ms, chủ yếu do thời gian lọc nhiễu. Việc giữ nguyên trạng thái cháy khi mất kết nối là lựa chọn thiên về an toàn: hệ thống không tự đóng cổng khi không còn biết lửa đã tắt hay chưa.

4.7. UC07. Báo ngập và ngừng nhận xe

Use case này mô phỏng tình huống nước dâng trong tầng hầm. Mục tiêu là cảnh báo kịp thời và không để thêm xe đi vào vùng ngập, trong khi xe đang trong bãi vẫn ra được. Đặc tả được trình bày ở Bảng 4.8.

Bảng 4.8: Đặc tả UC07. Báo ngập và ngừng nhận xe

Mục	Nội dung
Tác nhân chính	Sensor Client
Tác nhân phụ	Người vận hành
Mục tiêu	Báo động và không cho thêm xe vào khi mực nước vượt ngưỡng
Tiền điều kiện	Sensor Client đang gửi dữ liệu
Kích hoạt	Mực nước đo được đạt 20 mm

Mục	Nội dung
Luồng chính	– Máy chủ bật cảnh báo ngập: còi kêu, ô mực nước trên màn hình chuyển đỏ kèm dòng `FLOOD-NO ENTRY`, trang web hiển thị `FLOOD WARNING`. – Các lần quẹt thẻ ở cổng vào bị từ chối; các lần quẹt ở cổng ra vẫn được xử lý theo UC03. – Khi mực nước giảm xuống 15 mm, máy chủ tắt cảnh báo và nhận xe trở lại.
Luồng ngoại lệ	– Giá trị mực nước âm hoặc vượt 10 000 mm: bản tin bị từ chối, trạng thái cũ được giữ.
Hậu điều kiện	Không có xe mới vào bãi trong thời gian ngập

Sensor Client đọc ADC 10 bit ở chân A0 mỗi 250 ms và quy đổi tuyến tính sang milimét:

$$h = \begin{cases} 0, & r \le 20 \\ \dfrac{\min(r, 700) - 20}{700 - 20} \cdot 40 \text{ mm}, & r > 20 \end{cases} \tag{4.3}$$

trong đó r là giá trị ADC, 20 là mức nhiễu khi cảm biến khô, 700 là giá trị khi ngập toàn bộ vùng đo và 40 mm là chiều dài vùng đo. Chu kỳ 250 ms được chọn vì đọc ADC quá dày ảnh hưởng tới Wi-Fi trên ESP8266, trong khi mực nước thay đổi chậm. Khoảng trễ 5 mm giữa ngưỡng bật 20 mm và ngưỡng tắt 15 mm ngăn cảnh báo bật tắt liên tục khi mực nước dao động quanh một ngưỡng.

4.8. UC08. Xem tình trạng bãi trên trang web

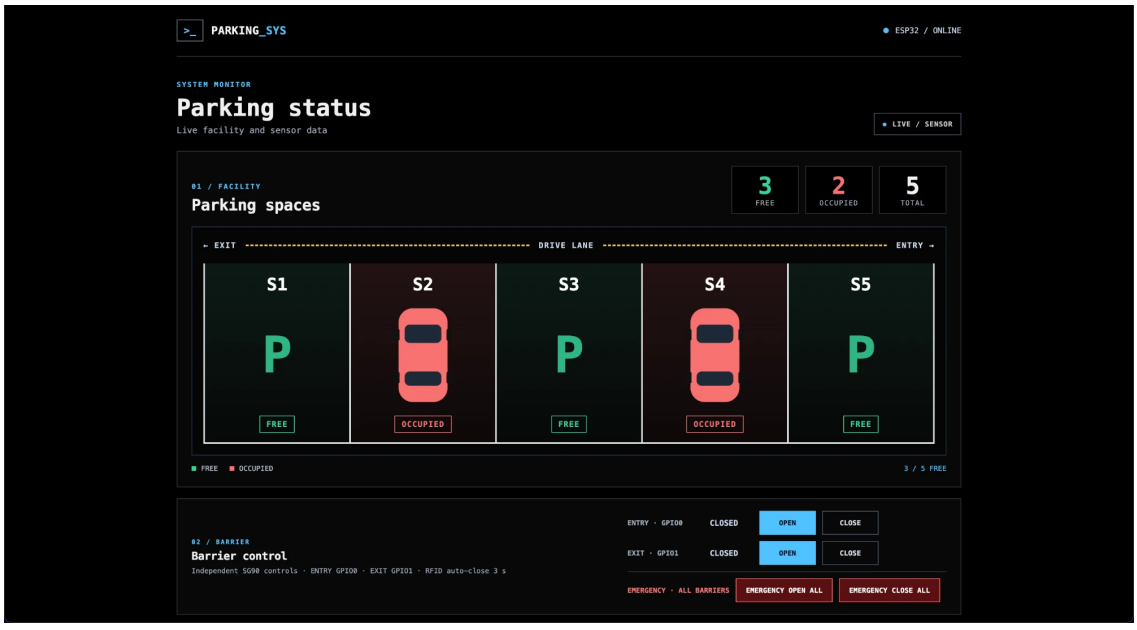
Use case này cho phép người vận hành nắm được toàn bộ trạng thái của bãi mà không cần có mặt tại cổng. Đặc tả được trình bày ở Bảng 4.9.

Bảng 4.9: Đặc tả UC08. Xem tình trạng bãi trên trang web

Mục	Nội dung
Tác nhân chính	Người vận hành
Tác nhân phụ	Sensor Client, RFID Client
Mục tiêu	Biết trạng thái ô đỗ, nhiệt độ, mực nước, cảm biến lửa và thanh chắn từ xa

Mục	Nội dung
Tiền điều kiện	Thiết bị của người vận hành ở cùng mạng Wi-Fi với máy chủ
Kích hoạt	Người vận hành mở trang `parking.local`
Luồng chính	– Thiết bị phân giải tên bằng mDNS và tải trang web nhúng trong firmware. – Trang web gọi `GET /api/parking` mỗi giây và `GET /api/barrier` khi mở trang. – Trang web vẽ sơ đồ năm ô đỗ, số chỗ trống, nhiệt độ, mực nước, trạng thái lửa và hai thanh chắn.
Luồng ngoại lệ	– Sensor Client mất kết nối quá 3 s: trang web báo `SENSOR OFFLINE` và làm mờ sơ đồ bãi. – Máy chủ mất kết nối: trang web chuyển từ `ESP32 / ONLINE` sang `OFFLINE`. – Thiết bị không hỗ trợ mDNS: người vận hành dùng địa chỉ IP của máy chủ.
Hậu điều kiện	Thông tin trên trang web trễ không quá khoảng 1 s so với thực tế

Hình 4.8 là ảnh chụp phần đầu trang web khi bãi có hai xe.



Hình 4.8: Trang web quản lý: sơ đồ bãi và điều khiển thanh chắn

Trong Hình 4.8, phần chính là sơ đồ năm ô đỗ theo đúng bố cục của mô hình, với làn xe, hướng vào và hướng ra; ô có xe được vẽ hình xe màu đỏ, ô trống hiện chữ P màu xanh.

Ba ô số ở góc trên phải cho biết số chỗ trống, số chỗ có xe và tổng số chỗ, trùng khớp với màn hình TFT cùng thời điểm ở Hình 3.3d. Phần dưới là các nút điều khiển của UC05.

Nhiệt độ hiển thị trên trang web và màn hình đến từ RFID Client. Client đọc cảm biến DHT11 mỗi 5 s, vì DHT11 cần tối thiểu 1 s giữa hai lần đọc và nhiệt độ môi trường thay đổi chậm, rồi gửi bản tin `climate` độc lập với việc có xe quẹt thẻ hay không. Nếu DHT11 trả về giá trị lỗi, client giữ giá trị hợp lệ gần nhất thay vì gửi số sai. Trang web được viết bằng HTML, CSS và JavaScript thuần, dùng phương pháp hỏi vòng thay vì WebSocket để máy chủ không phải giữ trạng thái kết nối cho mỗi trình duyệt.

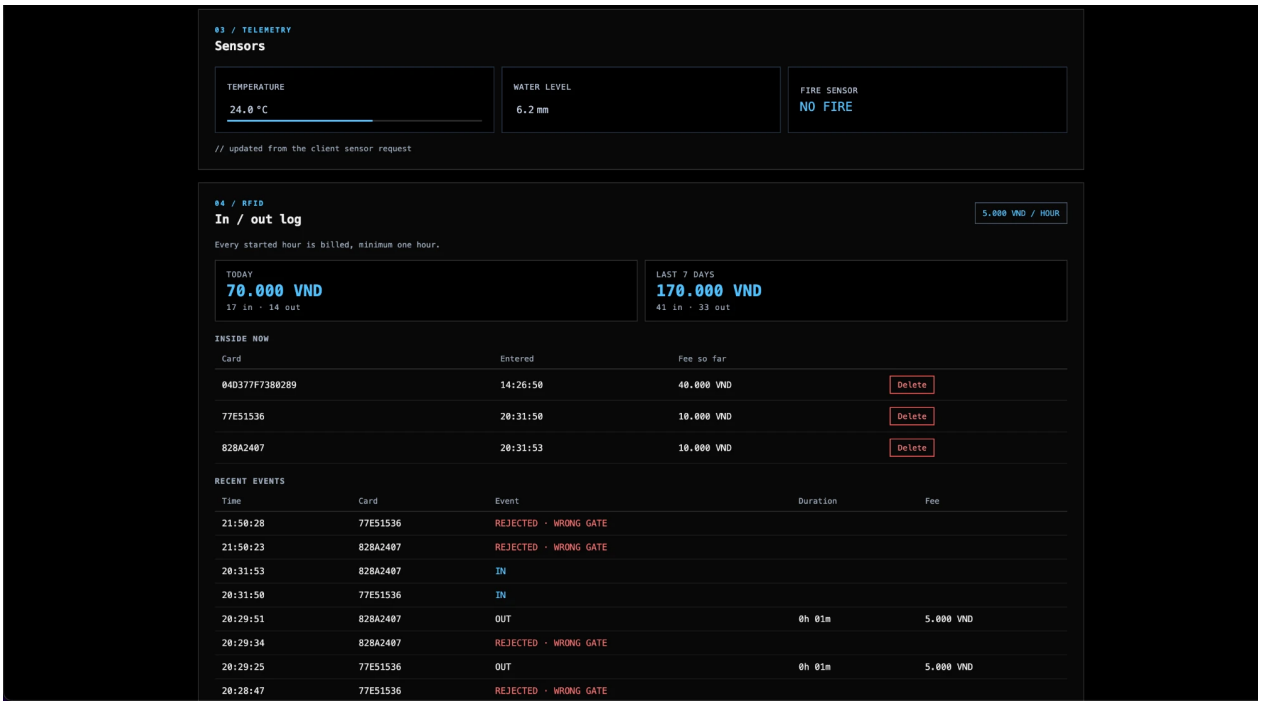
4.9. UC09. Tra cứu nhật ký và doanh thu

Use case này cung cấp cho người vận hành công cụ đối soát lượt xe và tiền thu được. Đặc tả được trình bày ở Bảng 4.10.

Bảng 4.10: Đặc tả UC09. Tra cứu nhật ký và doanh thu

Mục	Nội dung
Tác nhân chính	Người vận hành
Mục tiêu	Đối soát xe đang trong bãi, các sự kiện gần nhất và doanh thu
Tiền điều kiện	Thiết bị của người vận hành ở cùng mạng Wi-Fi với máy chủ
Kích hoạt	Người vận hành mở phần nhật ký trên trang web
Luồng chính	– Trang web gọi `GET /api/rfid` mỗi 2 s. – Máy chủ trả đơn giá, danh sách xe trong bãi kèm giờ vào và phí tạm tính, 10 sự kiện gần nhất và doanh thu. – Trang web hiển thị doanh thu, số lượt vào và ra trong ngày và trong 7 ngày gần nhất.
Luồng ngoại lệ	– Máy chủ vừa khởi động lại sau mất điện: dữ liệu được khôi phục từ flash; khi trình duyệt gửi giờ thực, các mốc thời gian được hiệu chỉnh và thời gian mất điện được bù.
Hậu điều kiện	Người vận hành có số liệu để đối chiếu với tiền thực thu

Hình 4.9 là phần trang web tương ứng với use case này.



Hình 4.9: Trang web quản lý: cảm biến, doanh thu và nhật ký vào ra

Hình 4.9 gồm bốn khối. Khối cảm biến hiển thị nhiệt độ, mực nước và trạng thái cảm biến lửa. Khối doanh thu cho biết tổng tiền, số lượt vào và số lượt ra trong ngày theo giờ Việt Nam và trong 7 ngày gần nhất. Bảng `INSIDE NOW` liệt kê các xe đang trong bãi; bảng `RECENT EVENTS` liệt kê 10 sự kiện gần nhất gồm xe vào, xe ra kèm thời lượng và phí, và các lần quẹt bị từ chối. Nhật ký được lưu trong bộ đệm vòng 10 phần tử, dưới 1 KB RAM và không dùng heap.

Toàn bộ phiên gửi xe, nhật ký, doanh thu và đồng hồ được lưu vào vùng NVS trên flash sau mỗi lần quẹt thẻ và định kỳ 5 phút. Với khoảng 300 lần ghi mỗi ngày và cơ chế rải đều lượt ghi của NVS, tuổi thọ flash không bị ảnh hưởng đáng kể. Máy chủ không có đồng hồ thời gian thực; trình duyệt của người vận hành gửi giờ thực qua `POST /api/time`, và máy chủ dời các mốc thời gian cũ theo đó.

4.10. UC10. Hiệu chỉnh danh sách xe trong bãi

Trong vận hành thực tế, danh sách xe trong bãi có thể lệch với thực tế, chẳng hạn khi xe đã rời bãi trong lúc sơ tán cháy mà không quẹt thẻ ra, hoặc khi khách làm mất thẻ. Use case này cho phép người vận hành sửa danh sách; đặc tả được trình bày ở Bảng 4.11.

Bảng 4.11: Đặc tả UC10. Hiệu chỉnh danh sách xe trong bãi

Mục	Nội dung
Tác nhân chính	Người vận hành
Mục tiêu	Xóa một xe không còn trong bãi khỏi danh sách
Tiền điều kiện	Xe có trong danh sách `INSIDE NOW`; người vận hành có mã PIN quản trị
Kích hoạt	Người vận hành bấm `Delete` ở dòng của xe và xác nhận
Luồng chính	– Trang web gửi `POST /api/rfid/remove` kèm mã thẻ và mã PIN. – Máy chủ xóa phiên của thẻ và hủy lượt thanh toán đang chờ của xe đó nếu có. – Danh sách được lưu lại vào flash; trang web cập nhật bảng `INSIDE NOW`.
Luồng ngoại lệ	– Thẻ không có trong danh sách: máy chủ trả mã 404. – Sai hoặc thiếu mã PIN: máy chủ trả mã 401.
Hậu điều kiện	Thẻ có thể dùng để gửi xe lại; không có phí, hóa đơn hay thay đổi doanh thu; thanh chắn không chuyển động

Theo Bảng 4.11, thao tác xóa được thiết kế khác hẳn UC04: không tính phí và không mở thanh chắn, vì mục đích là sửa dữ liệu chứ không phải cho xe ra. Sau khi xóa, số thẻ trong bãi khớp lại với số ô có xe theo cảm biến, nên cảnh báo xe không quẹt thẻ của UC02 và điều kiện bãi đầy được tính đúng trở lại. Do thao tác không để lại dấu vết trong nhật ký, việc yêu cầu mã PIN là bắt buộc.

CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày phương pháp và kết quả kiểm thử ở hai mức, kiểm thử tự động trên máy tính và kiểm thử hệ thống trên mô hình, sau đó đánh giá hiệu năng, tài nguyên và các hạn chế còn tồn tại.

5.1. Phương pháp kiểm thử

Lỗi trong bộ đọc bản tin, cách tính phí hay logic lọc nhiễu rất khó tái hiện trên phần cứng vì phụ thuộc chính xác vào thời điểm và chuỗi byte. Do đó, nhóm tổ chức kiểm thử ở hai mức. Mức thứ nhất là kiểm thử tự động trên máy tính: các module nghiệp vụ của máy chủ được biên dịch cùng hai tệp header giả thay cho vùng găng và đồng hồ; mã nguồn của hai bo thu thập được biên dịch nguyên vẹn cùng một lớp giả lập thư viện Arduino gồm đồng hồ điều khiển được, mức logic từng chân, ADC, Wi-Fi, UDP, đầu đọc MFRC522 và DHT11. Mỗi gói tin mà client giả lập gửi ra đều được đưa vào chính bộ đọc JSON của máy chủ, bảo đảm hai phía luôn thống nhất định dạng. Mức thứ hai là các kịch bản kiểm thử trên mô hình thật, mỗi use case có ít nhất một kịch bản.

5.2. Kết quả kiểm thử tự động

Bộ kiểm thử gồm 7 nhóm, chạy bằng lệnh `make -C tests` trong vài giây. Bảng 5.1 tóm tắt nội dung và kết quả của từng nhóm.

Bảng 5.1: Kết quả kiểm thử tự động

Nhóm	Nội dung kiểm tra chính	Kết quả
`payment`	CRC16 với giá trị chuẩn; chuỗi VietQR trùng từng byte với mã mẫu; từ chối tài khoản, số tiền không hợp lệ; chỉ chấp nhận giao dịch đúng mã và đúng số tiền	38/38
`json_parsers`	Bản tin hợp lệ với mọi thứ tự khóa; 46 dạng bản tin sai đều bị từ chối; biên của trường `seq`	78/78
`rfid_session`	Phí theo giờ tại các biên; vào, ra, sai cổng, bãi đầy; nhật ký vòng; lưu và khôi phục sau mất điện; đặt giờ; ngập có trễ; doanh thu theo ngày; chống xử lý trùng; luồng thanh toán QR; xóa xe; cảnh báo xe không thẻ	181/181
`gate_hold`	Giữ thanh chắn khi tia bị che, chỉ đóng sau 1 s thông suốt liên tục, đếm lại khi tia bị che giữa chừng	10/10
`gif_frames`	Giải nén đủ 28 khung ảnh động, từ chối yêu cầu sai kích thước	89/89
`sensor_client`	Chạy mã Sensor Client: gửi định kỳ 1 s, gửi ngay sau 50 ms, lọc nhiễu, báo cháy, quy đổi mực nước, mất Wi-Fi	18/18
`rfid_client`	Chạy mã RFID Client: định dạng UID, chống đọc lặp 2 s, gửi `climate`, gửi lại khi chưa có xác nhận, bỏ cuộc sau 6 lần	25/25
Tổng		439/439

Kết quả trong Bảng 5.1 cho thấy toàn bộ 439 phép kiểm tra đều đạt. Nhóm `rfid_session` chiếm tỉ trọng lớn nhất vì chứa phần lớn logic nghiệp vụ, đặc biệt là luồng thanh toán với nhiều ngoại lệ. Bộ kiểm thử cũng đã phát hiện một lỗi nằm trong chính bài kiểm thử: khi kiểm tra bãi đầy, một thẻ từ bài kiểm thử trước vẫn còn trong bãi khiến chỉ còn 4 chỗ trống; bài kiểm thử được sửa để tự dọn trạng thái trước khi chạy.

5.3. Kết quả kiểm thử trên mô hình

Các kịch bản kiểm thử hệ thống được thực hiện trên mô hình thật với cả ba bo mạch. Bảng 5.2 liệt kê các kịch bản chính theo thứ tự use case, cùng hành vi mong đợi mà nhóm đã quan sát được.

Bảng 5.2: Kịch bản kiểm thử trên mô hình và kết quả quan sát

UC	Thao tác	Kết quả
UC01	Đặt xe vào ô S2	Ô S2 trên màn hình chuyển đỏ gần như tức thì
UC01	Đưa vật nhanh qua cảm biến dưới 50 ms	Trạng thái không đổi
UC01	Rút điện Sensor Client	Sau khoảng 3 s màn hình báo `SENSOR OFFLINE`
UC02	Quẹt thẻ mới ở cổng vào	Thanh chắn mở, đóng sau 3 s; nhật ký có dòng `IN`
UC02	Giữ vật chắn tia IR cổng vào quá 3 s	Thanh chắn giữ mở, chỉ đóng sau khi bỏ vật chắn khoảng 1 s
UC02	Gửi lặp lại một gói quẹt thẻ cùng `seq`	Hai lần đều nhận xác nhận, cổng chỉ mở một lần
UC02	Quẹt lại thẻ đã vào ở cổng vào	Không mở, báo `WRONG GATE` trong 5 s
UC02	Quẹt thẻ thứ 6 khi đã có 5 xe	Không mở, báo `PARKING FULL`
UC02	Đặt xe vào ô mà không quẹt thẻ	Sau 15 s còi kêu ngắt quãng, màn hình báo `CAR W/O CARD`
UC03	Quẹt ra, chuyển khoản bằng ứng dụng ngân hàng	Màn hình hiện mã QR; vài giây sau khi chuyển, cổng ra mở, có hóa đơn và dòng `OUT`
UC03	Chuyển dư 1.000 VNĐ, sau đó chuyển đúng phí	Lần đầu cổng vẫn đóng, lần sau cổng mở
UC04	Quẹt ra, không chuyển khoản, sau đó bấm `Confirm paid`	Sau 180 s màn hình báo `PAY AT BOOTH`; xác nhận thì cổng mở, có hóa đơn
UC05	Bấm `OPEN`, `CLOSE` của từng cổng	Thanh chắn tương ứng chuyển động, trạng thái trên web đúng
UC05	Bấm `CLOSE` khi đang che tia IR cổng	Trang web báo `CAR IN GATE`, thanh chắn giữ mở
UC06	Đưa lửa gần cảm biến, bấm `CLOSE` trên web	Còi kêu, hai thanh chắn mở; lệnh đóng bị từ chối
UC07	Nhúng cảm biến nước sâu hơn 20 mm	Còi kêu, `FLOOD WARNING`; thẻ mới bị từ chối, xe trong bãi vẫn ra được
UC08	Mở `parking.local` trên điện thoại, để yên 1 phút	Sơ đồ bãi khớp màn hình TFT; nhiệt độ vẫn cập nhật khi không có xe quẹt thẻ
UC09	Gửi rồi lấy 2 xe, rút điện máy chủ 1 phút rồi cấp lại	Doanh thu và nhật ký đúng; xe còn trong bãi vẫn ở danh sách sau khi cấp điện lại

UC	Thao tác	Kết quả
UC10	Bấm `Delete` ở một xe, nhập mã PIN	Xe rời danh sách, doanh thu và nhật ký không đổi, thanh chắn đứng yên

Các kịch bản trong Bảng 5.2 đều cho kết quả đúng như mong đợi. Kịch bản quan trọng nhất về an toàn là giữ thanh chắn khi có vật trong cổng: thanh chắn không hạ xuống dù hẹn giờ đã hết, cả với lệnh tự động lẫn lệnh thủ công. Kịch bản thanh toán được thực hiện với giao dịch thật qua ứng dụng ngân hàng, xác nhận rằng mã VietQR sinh trên chip được các ứng dụng đọc đúng số tiền và nội dung.

5.4. Đánh giá hiệu năng và tài nguyên

Bảng 5.3 ước tính độ trễ của các luồng chính từ thông số thiết kế, đối chiếu với chỉ tiêu ở Bảng 3.2.

Bảng 5.3: Độ trễ của các luồng chính

Luồng	Thành phần	Tổng
Xe vào ô, màn hình cập nhật	Lọc 50 ms, UDP vài ms, vẽ 23 ms	khoảng 0,1 s
Xe vào ô, trang web cập nhật	Như trên cộng chu kỳ hỏi tối đa 1 s	tối đa 1,1 s
Quẹt thẻ, thanh chắn mở	Quét đầu đọc 10 ms, UDP, xử lý dưới 1 ms, servo quay 90°	khoảng 0,2 s
Có lửa, còi kêu	Lọc 50 ms, UDP	khoảng 0,06 s
Mất một gói quẹt thẻ	Gửi lại sau 300 ms	khoảng 0,5 s

Theo Bảng 5.3, mọi luồng đều đạt chỉ tiêu: màn hình cập nhật khoảng 0,1 s so với yêu cầu 0,2 s, thanh chắn mở khoảng 0,2 s so với yêu cầu 0,5 s. Về tài nguyên, firmware máy chủ có kích thước 1 300 560 byte, chiếm khoảng 85% phân vùng ứng dụng 1,5 MB; RAM tĩnh dùng khoảng 31% trong số 321 KB, phần lớn RAM động dành cho bộ đệm khung 115 KB. Các con số này cho thấy ESP32-C3 đáp ứng được toàn bộ chức năng, nhưng dung lượng flash còn lại không đủ cho cơ chế nạp firmware qua mạng với hai phân vùng ứng dụng.

5.5. Hạn chế còn tồn tại

Bên cạnh các kết quả đạt được, hệ thống còn một số hạn chế, phần lớn là đánh đổi có chủ đích ở quy mô mô hình. Bảng 5.4 phân tích nguyên nhân và hướng khắc phục của từng hạn chế.

Bảng 5.4: Hạn chế, nguyên nhân và hướng khắc phục

Hạn chế	Nguyên nhân	Hướng khắc phục
Bản tin UDP không được ký, thiết bị trong mạng có thể gửi thẻ giả	Ưu tiên giao thức đơn giản trên mạng Wi-Fi riêng có WPA2	Ký bản tin bằng HMAC kèm số thứ tự chống phát lại
Trang web dùng HTTP, mã PIN có thể bị nghe lén trong mạng	HTTPS phía máy chủ cần thêm khoảng 40 KB RAM	Dùng chip có PSRAM như ESP32-S3
Mỗi lúc chỉ một xe thanh toán ở cổng ra	Một cổng ra, một màn hình; một số ví thay nội dung chuyển khoản	Mỗi lượt dùng một tài khoản ảo riêng
Chuyển sai số tiền không được hoàn tự động	Máy chủ chỉ đọc được giao dịch	Quy trình hoàn tiền của người vận hành
Nguồn client hạ áp bằng diode, điện áp thay đổi theo tải	Tận dụng linh kiện sẵn có	Thay bằng ổn áp LDO 3,3 V
Cảm biến cổng ra bị che khi cấp nguồn có thể làm sai chế độ khởi động	GPIO2 là chân strapping	Chuyển cảm biến sang chân không phải strapping trên bo nhiều chân hơn
Chưa có giờ thực trước khi trình duyệt kết nối	Máy chủ không có đồng hồ thời gian thực	Thêm module RTC hoặc đồng bộ giờ qua SNTP

Các hạn chế trong Bảng 5.4 không ảnh hưởng tới việc trình diễn và đánh giá chức năng trên mô hình, nhưng cần được giải quyết trước khi đưa hệ thống vào vận hành thực tế.

KẾT LUẬN

Đề tài đã xây dựng hoàn chỉnh một hệ thống bãi đỗ xe thông minh quy mô mô hình gồm ba vi điều khiển phối hợp qua Wi-Fi. Máy chủ ESP32-C3 chạy FreeRTOS quản lý phiên gửi xe, tính phí, điều khiển hai thanh chắn có cảm biến chống hạ xuống thân xe, hiển thị trên màn hình TFT và phục vụ trang web quản lý; hai bo ESP8266 thu thập trạng thái ô đỗ, cháy, mực nước, nhiệt độ và sự kiện quẹt thẻ. Cả 10 use case, từ gửi xe, thanh toán tới xử lý sự cố và quản lý bãi, đều được hiện thực và kiểm chứng; người lái thanh toán bằng mã VietQR với số tiền cố định, hệ thống tự đối soát giao dịch qua SePay trước khi mở cổng ra.

Về mặt học thuật, đề tài vận dụng các kiến thức trọng tâm của môn học: lập trình đa tác vụ và đồng bộ trên hệ điều hành thời gian thực, điều khiển ngoại vi bằng khối phần cứng chuyên dụng như PWM, SPI có DMA, ADC, thiết kế nguồn và ghép nối mức logic giữa các linh kiện, giao tiếp mạng không dây với cơ chế tin cậy ở tầng ứng dụng, quản lý bộ nhớ hạn chế, và kiểm thử phần mềm nhúng không cần phần cứng với 439 phép kiểm tra tự động.

Từ các hạn chế đã phân tích, nhóm đề xuất các hướng phát triển sau:

- quản lý danh sách thẻ đăng ký như vé tháng và khóa thẻ bị mất từ trang web;
- ghép thẻ với ô đỗ để biết chính xác xe nào đỗ ở ô nào;
- ký bản tin UDP bằng HMAC và dùng HTTPS cho trang quản trị;
- thay mạch hạ áp bằng diode bằng ổn áp LDO và bổ sung module đồng hồ thời gian thực;
- kết nối lên nền tảng đám mây qua MQTT để quản lý nhiều bãi, thống kê và đặt chỗ trước.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, *ESP32-C3 Series Datasheet* và *ESP32-C3 Technical Reference Manual*.

[2] Espressif Systems, *ESP-IDF Programming Guide* (v6.1): Wi-Fi, LEDC, SPI Master, LCD, NVS, HTTP Server, ESP Timer, FreeRTOS, Partition Tables.

[3] Espressif Systems, *mDNS component* (`espressif/mdns`), ESP Component Registry.

[4] Espressif Systems, *ESP8266EX Datasheet*.

[5] ESP8266 Community, *ESP8266 Arduino Core Documentation*.

[6] Sitronix, *ST7789V — 240RGB×320 dot 262K Color TFT Controller*.

[7] NXP Semiconductors, *MFRC522 Standard Performance MIFARE and NTAG Frontend*, Product Data Sheet.

[8] TowerPro, *SG90 Micro Servo Datasheet*.

[9] Aosong, *DHT11 Humidity & Temperature Sensor Datasheet*.

[10] R. Barry, *Mastering the FreeRTOS Real Time Kernel*, Real Time Engineers Ltd.

[11] S. Cheshire, M. Krochmal, *RFC 6762: Multicast DNS*, IETF, 2013.

[12] J. A. Storer, T. G. Szymanski, "Data compression via textual substitution", *Journal of the ACM*, 29(4), 1982.

[13] Thư viện *MFRC522* (miguelbalboa) và *DHT sensor library* (Adafruit) cho Arduino.

[14] NAPAS, *Đặc tả kỹ thuật mã QR chuyển tiền liên ngân hàng (VietQR)*, dựa trên EMVCo *QR Code Specification for Payment Systems — Merchant-Presented Mode*.

[15] SePay, *SePay Developer Documentation*: API giao dịch, tài khoản ảo (VA).

[16] Knowledge Sourcing Intelligence, *Smart Parking Market*. https://www.knowledge-sourcing.com/report/smart-parking-market

[17] Meticulous Research, *Smart Parking Market*. https://www.meticulousresearch.com/pressrelease/1052/smart-parking-market-2032

[18] KPS System, *Giải pháp bãi giữ xe thông minh*. https://kps.com.vn/en/tin-tuc/giai-phap-bai-giu-xe-thong-minh-399.html

[19] Hikvision, *Parking Lots* và *On-street Parking*. https://www.hikvision.com/vn/solutions/solutions-by-scenario/parking-lots, https://www.hikvision.com/vn/products/parking-management/on-street-parking/

[20] Nedap, *Wireless vehicle detection system SENSIT*. https://www.parking.net/parking-showcases/nedap-identification-systems/nedap-wireless-vehicle-detection-system-sensit-accurate-parking-occupancy-detection-system

[21] SKIDATA, *The Benefits of Parking Guidance for Parking Operators*. https://www.skidata.com/en-nordics/skidata-blog/parking-guidance-benefits-for-parking-operators

[22] FPT Shop, *Có phần mềm giữ xe miễn phí nào không?* https://fptshop.com.vn/tin-tuc/danh-gia/phan-mem-giu-xe-mien-phi-172797

[23] Việt Nam News, *Hà Nội pilots iParking for cars from May* (28/3/2017) và *Hà Nội expands iParking in July*. https://vietnamnews.vn/society/373668/ha-noi-pilots-iparking-for-cars-from-may.html, https://vietnamnews.vn/society/449045/ha-noi-expands-iparking-in-july.html

[24] VietNamNet, *Hà Nội tạm dừng thí điểm mô hình trông giữ xe iParking từ 1/9*, 31/8/2020. https://vietnamnet.vn/ha-noi-tam-dung-thi-diem-mo-hinh-trong-giu-xe-iparking-tu-19-i262776.html

[25] Mekong ASEAN, *Từ 5/5 sẽ thu phí không dừng tại 5 sân bay lớn*. https://mekongasean.vn/tu-55-se-thu-phi-khong-dung-tai-5-san-bay-lon-28091.html

[26] Tạp chí Luật sư Việt Nam, *Hà Nội sẽ thu phí điện tử không dừng toàn bộ bãi đỗ xe từ ngày 01/7*, 25/5/2026. https://lsvn.vn/ha-noi-se-thu-phi-dien-tu-khong-dung-toan-bo-bai-do-xe-tu-ngay-01-7-a173253.html

[27] Capterra, *SKIDATA*. https://www.capterra.com/p/141629/SKIDATA/alternatives/

PHỤ LỤC A. ĐÓNG GÓP CỦA THÀNH VIÊN VÀ PHÂN CHIA ĐIỂM

A.1. Phân công công việc

Đề tài có ba mảng với khối lượng tương đương: phần mềm gồm firmware ba bo mạch, trang web và kiểm thử; điện gồm chọn linh kiện, thiết kế nguồn và đi dây; cơ khí gồm mô hình và cơ cấu thanh chắn. Mỗi thành viên phụ trách chính một mảng và tham gia phần giao với các mảng khác. Nhóm làm việc trên một kho GitHub chung và họp mỗi tuần để tích hợp, chạy thử trên mô hình. Bảng A.1 tóm tắt phân công.

Bảng A.1: Phân công công việc

Thành viên	Mảng chính	Phần giao với mảng khác
Nguyễn Chí Minh	Firmware máy chủ: kiến trúc tác vụ, UDP, HTTP, màn hình, thanh toán VietQR và SePay, bảo mật; trang web; bộ kiểm thử tự động	Thống nhất giao thức với hai client; tích hợp và tối ưu toàn hệ thống; các sơ đồ thiết kế
Đinh Đức Thịnh	Chọn linh kiện, thiết kế nguồn với tụ lọc và diode hạ áp, sơ đồ chân, sơ đồ nguyên lý, đi dây	Firmware Sensor Client và RFID Client; khởi tạo dự án ESP-IDF
Nguyễn Việt Tiến	Mô hình bãi xe, hộp điều khiển, cơ cấu thanh chắn và cảm biến cổng	Firmware thanh chắn và còi; kịch bản kiểm thử trên mô hình; tài liệu cài đặt

Theo Bảng A.1, phần việc điện và cơ khí như đo đạc, đi dây, sửa lỗi nguồn, chế tạo mô hình và căn chỉnh servo chiếm nhiều thời gian nhưng không thể hiện trong lịch sử Git. Vì vậy nhóm đánh giá đóng góp theo mảng việc và thời gian thực hiện, không theo số dòng mã. Ngoài ra, cả nhóm cùng phân tích yêu cầu, thống nhất giao thức, tinh chỉnh thông số trên mô hình như thời gian lọc nhiễu và thời gian mở cổng, và rà soát chéo báo cáo.

A.2. Tỉ lệ đóng góp và phân chia điểm

Cả ba thành viên thống nhất rằng mức đóng góp là ngang nhau. Theo quy định của môn học, toàn bộ thành viên nhận 100% số điểm của báo cáo.

Bảng A.2: Tỉ lệ đóng góp và tỉ lệ điểm

Thành viên	MSSV	Tỉ lệ đóng góp	Tỉ lệ điểm báo cáo
Nguyễn Chí Minh	24020232	33,3%	100%
Đinh Đức Thịnh	24020313	33,3%	100%
Nguyễn Việt Tiến	24020322	33,3%	100%

Xác nhận của các thành viên:

Nguyễn Chí Minh	Đinh Đức Thịnh	Nguyễn Việt Tiến
_____	_____	_____